FAAV Build Manual

MIT AURA Lab — 2026 v4.0

MIT AURA Lab

Contents

1 FAAV Build Manual . . . 9
1.1 How this manual is organised . . . 9
1.2 Semester map . . . 9
1.3 Conventions . . . 10
1.4 Versioning . . . 10

I Build modules

2 Module 1 — Orientation . . . 13
2.1 The robot . . . 13
2.2 Mechanical design . . . 13
2.3 Robot operation in one paragraph . . . 13
2.4 1-225 Lab rules . . . 13
2.5 Lab use . . . 14

3 Module 2 — Motor and transmission shaft . . . 15
3.1 Summary . . . 15
3.2 Requirements . . . 15
3.3 The motor - AS2303 KV1500 . . . 15
3.4 Motor shaft . . . 16
3.5 Sensor magnet . . . 17
3.6 Preparing the robot bottom plate . . . 17
3.7 Mounting the motor . . . 17
3.8 Solder the motor wires to the Electronic Speed Controller . . . 18

4 Module 3 — Motor driver, sensor and serial connection . . . 19
4.1 Summary . . . 19
4.2 Requirements . . . 19
4.3 Electrical connections . . . 20
4.4 Electronic Speed Controller . . . 21
4.5 Flight controller microcontroller . . . 21
4.6 Encoder (or Angular sensor) . . . 22
4.7 Powering the encoder . . . 23
4.8 Mounting . . . 24

5 Module 4 — Closed-loop control and tuning . . . 25
5.1 Summary . . . 25
5.2 Requirements . . . 25

5.3 Electronics check . . . 25
5.4 Firmware . . . 25
5.5 Controlling the motor from a laptop . . . 25
5.6 Commanding and monitoring the ESC . . . 26
5.7 Tuning the cascaded controller . . . 27
5.8 Calibration . . . 29
5.9 Why we use an ESC that we reprogram . . . 29

6 Module 5 — Frame, crank, linkage and gears . . . 31
6.1 Summary . . . 31
6.2 Requirements . . . 31
6.3 Main body . . . 31
6.4 Reduction . . . 31
6.5 Crank and linkage . . . 32
6.6 Gears and shafts . . . 33
6.7 Linkage-Gear assembly . . . 33
6.8 Mount the right wing . . . 34
6.9 Mount the left wing . . . 34
6.10 Insert the front plate . . . 34

7 Module 6 — Flapping simulation (Ptera) . . . 37
7.1 Summary . . . 37
7.2 Requirements . . . 37
7.3 What Ptera does . . . 37
7.4 Installing Ptera . . . 38
7.5 How a Ptera model is put together . . . 38
7.6 Modelling the tail . . . 38
7.7 Beyond prescribed motion . . . 39

8 Module 7 — Wing fabrication . . . 43
8.1 Summary . . . 43
8.2 Requirements . . . 43
8.3 Wing roots . . . 43
8.4 Example and tips for building the wing . . . 43
8.5 Assembly . . . 44

9 Module 8 — Tail . . . 45
9.1 Summary . . . 45
9.2 Reduction installation . . . 45
9.3 Tail . . . 46
9.4 Trailing edge holder . . . 46

10 Module 9 — Flight controller, PCB and dry testing . . . 49
10.1 Summary . . . 49

10.2 Requirements . . . 49
10.3 Flight controller . . . 49
10.4 Connect battery . . . 49
10.5 Dry testing . . . 49

11 Module 10 — Waterproofing . . . 51
11.1 Summary . . . 51
11.2 Requirements . . . 51
11.3 Flight controller waterproofing . . . 51
11.4 ESC waterproofing . . . 51
11.5 Battery waterproofing . . . 52
11.6 Battery connector waterproofing . . . 52
11.7 Spend time discussing COT and measurement . . . 52
11.8 Gearbox . . . 52

12 Module 11 — Wet testing . . . 53
12.1 Summary . . . 53
12.2 Requirements . . . 53
12.3 Pre-immersion checks . . . 53
12.4 The tank . . . 53
12.5 What to log . . . 54
12.6 Housing . . . 54
12.7 Trim . . . 54
12.8 Cost of transport . . . 55

13 Module 12 — Competition . . . 57
13.1 Summary . . . 57
13.2 The competition rules . . . 57
13.3 Requirements . . . 57
13.4 Feedback . . . 57

II Reference

14 Parts list . . . 61
14.1 . . . 62

15 Brushless motors . . . 63
15.1 Start with the brushed DC motor . . . 63
15.2 What changes with a brushless motor . . . 64
15.3 Thermal limits . . . 64
15.4 Why an outrunner . . . 65

16 The G431 ESC board . . . 67
16.1 What it is . . . 67

16.2 The parts that matter . . . 67
16.3 How the three phases are generated . . . 68
16.4 What it can drive . . . 68
16.5 Why there is a daughterboard . . . 68
16.6 What we do not use . . . 69

17 How SimpleFOC works . . . 71
17.1 The problem it solves . . . 71
17.2 Three objects, and what links them . . . 71
17.3 The two loops . . . 71
17.4 What the inner loop actually computes . . . 71
17.5 What the outer loop computes . . . 73
17.6 Alignment, and why the motor twitches at power-on . . . 73
17.7 The main ESC code . . . 73
17.8 Where to look next . . . 74

18 Software . . . 75
18.1 ESC . . . 75
18.2 External documentation . . . 75

19 Design notes . . . 77
19.1 Log of 2026 changes . . . 77
19.2 Gearbox corrosion . . . 77

20 Instructor checklist . . . 79
20.1 Module 1 — Orientation . . . 79
20.2 Module 2 — Motor and transmission shaft . . . 79
20.3 Module 3 — Motor driver, sensor and serial connection . . . 79
20.4 Module 4 — Closed-loop control and tuning . . . 79
20.5 Module 5 — Frame, crank, linkage and gears . . . 80
20.6 Module 6 — Flapping simulation (Ptera) . . . 80
20.7 Module 7 — Wing fabrication . . . 80
20.8 Module 8 — Tail . . . 81
20.9 Module 9 — Flight controller, PCB and dry testing . . . 81
20.10 Module 10 — Waterproofing . . . 81
20.11 Module 11 — Wet testing . . . 82
20.12 Module 12 — Competition . . . 82

21 Changelog . . . 83
21.1 fall-2026 . . . 83

4.1 AS5047P in 3.3V mode: VDD tied to VDD3V3, bypassing the internal LDO . . . 24
4.2 AS5047 breakout board . . . 24
4.3 Different AS5047 breakout, with the pads labelled CSN, SCK, MISO, MOSI, GND and 5V. 24
6.1 Transmission assembled . . . 33

6.2 Crank-linkage 33
8.1 Strips laid out and bonded over the template 44
8.2 Trimming the membrane to the outline 44
9.1 Faulhaber gearbox 45

1. FAAV Build Manual

MIT AURA Lab — Fall 2026

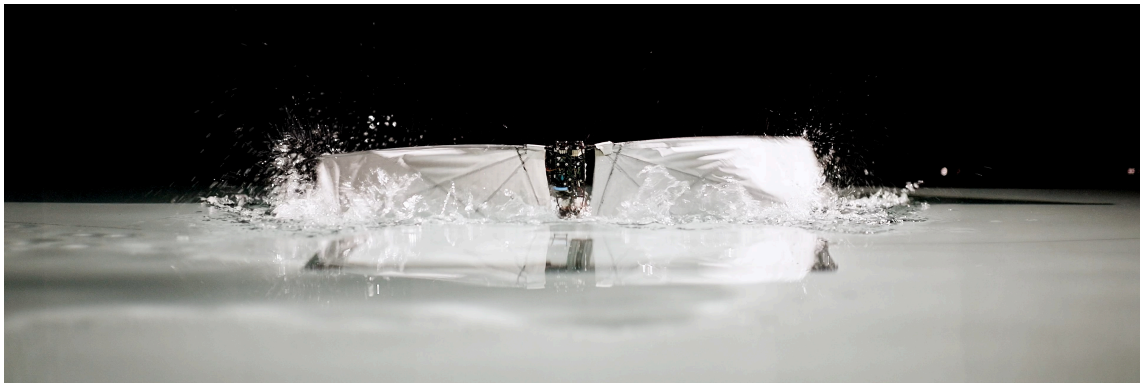


Figure 1.1: The flapping-wing aerial-aquatic robot (FAAV)

Over the semester you build a **flapping vehicle**: a robot that with the right design can fly and swim by flapping the same wings. The aim of this project is to demonstrate swimming behavior. This manual is the build side of the course. It is sequenced to the weekly modules, and each page covers exactly what you do in that module.

1.1 How this manual is organised

Build modules are cumulative and in order.

Reference holds the material you consult rather than follow: the parts list, the firmware repositories, and the design notes explaining why the robot is the way it is.

1.2 Semester map

Module	Week	Build	Paired lecture
1	Sep 9		Intro to robotic locomotion
2	Sep 14	Orientation, teams, the robot. Motor and transmission shaft	Reaction forces in fluids
3	Sep 21	Motor driver, sensor, serial	Bird-scale flapping design
4	Sep 28	Closed-loop control and tuning	Swimming robots I
5	Oct 5	Frame, crank, linkage, gears	Undulatory locomotion
—	Oct 12	*No module*	—

Module	Week	Build	Paired lecture
6	Oct 19	Flapping simulation (Ptera)	Insect-scale flapping
7	Oct 26	Wing fabrication	Swimming robots II
8	Nov 2	Tail and housing	Multi-modal locomotion
9	Nov 9	Flight controller, PCB, dry testing	The future of actuation
10	Nov 16	Waterproofing	Multirotors
11	Nov 23	Wet testing	Fixed-wing flyers
12	Nov 30	Competition	Pair presentations

1.3 Conventions

> ⚠ Warning
>
> An operation whose likely failure mode destroys a part or injures you. Read it twice before acting.

> ℹ Checkpoint
>
> Do not proceed past this without having completed all the tasks listed.

> Why
>
> Design rationale.

1.4 Versioning

This is version **fall-2026**. Corrections published during the semester are dated in the Changelog. If a step here contradicts what we discuss, report it to Raph on slack so the next student does not lose time to it.

I Build modules

2 Module 1 — Orientation 13

3 Module 2 — Motor and transmission shaft 15

4 Module 3 — Motor driver, sensor and serial connection 19

5 Module 4 — Closed-loop control and tuning 25

6 Module 5 — Frame, crank, linkage and gears 31

7 Module 6 — Flapping simulation (Ptera) 37

8 Module 7 — Wing fabrication 43

9 Module 8 — Tail 45

10 Module 9 — Flight controller, PCB and dry testing 49

11 Module 10 — Waterproofing 51

12 Module 11 — Wet testing 53

13 Module 12 — Competition 57

2. Module 1 — Orientation

2.1 The robot

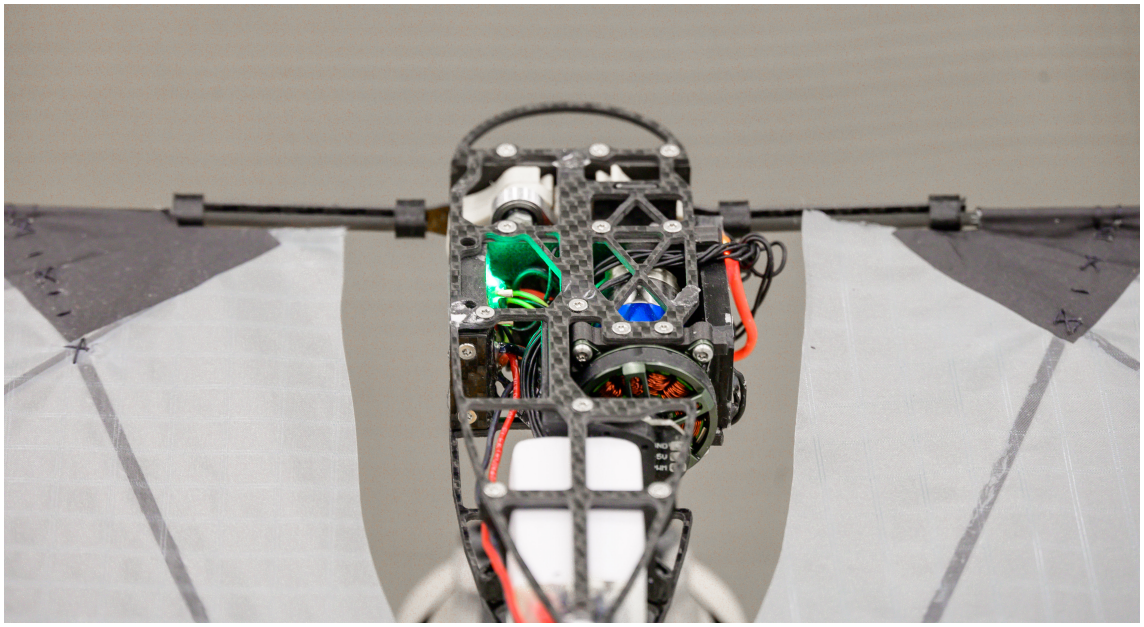


Figure 2.1: The flapping robot

We will develop, plan and optimize a vehicle with powerful custom actuation for flapping locomotion. The build divides into four subsystems:

1. **Transmission** - motor, custom shaft, gearbox, crank and linkage (Modules 2, 5)
2. **Electronics** - ESC, angular sensor, flight controller (Modules 2, 3, 4)
3. **Aero/hydro surfaces** - wings, tail (Modules 6-8)
4. **Waterproofing and testing** - potting, immersion, tuning (Modules 10–12).

2.2 Mechanical design

The 3D design (CAD) of the core of the robot is available here: https://a360.co/3UVNc2T

2.3 Robot operation in one paragraph

One 4S battery (16.8 V max) feeds an custom ESC that steps VBAT down to 5 V to power the flight controller and the encoder. The flight controller commands the ESC over serial. The ESC drives a brushless motor through a 23.04:1 gearbox, and a crank–linkage converts that rotation into undulation motion.

2.4 1-225 Lab rules

Every component, robot and tool that is required is in the cabinet marked “S.2986 Class”.

! All parts, components, tools belong to the MIT AURA Lab

If you would like to use anything outside of what is marked as class materials, please ask.

2.4.1 Safety

Our safety officer is Ethan Fulcher.

- **Assembly**. Wear goggles for mechanical assembly, particularly important as we are handling small parts. Find them in the "safety" drawer
- **Soldering**. On the fume extraction bench only.
- **Tank**. Absolutely no swimming or going into the tank. Use the poles/grippers to recover robots and parts.
- **Keep water off the floor.** Water on the floor sets off an alarm, which ends the session for everyone.

2.5 Lab use

- This is a research lab, many other projects are happening at the same time. Please be respectful.
- Wednesday only

3. Module 2 — Motor and transmission shaft

3.1 Summary

A brushless motor with a custom shaft pressed into its rotor bell, with the sun gear and the sensor magnet installed, bolted to its bracket and mounted on the motor plate. The motor is soldered to the electronic speed controller. The gearbox itself goes on in Module 4.

3.2 Requirements

Parts	AS2303 KV1500 motor, custom motor shaft, B-G431B-ESC1 board, 6 × 2.5 mm diametric magnet, 3D-printed parts: 1) magnet bracket, 2) vertical motor bracket, 3) ESC box. Bottom fuselage plate, mounting screws
Tools	Circlip pliers, manual arbor press, superglue, torx drivers
Read	This page, and the *Motor shaft* section below in full before touching the motor. Brushless motors for the background

3.3 The motor - AS2303 KV1500

The AS2303 is a **brushless outrunner**. Read Brushless motors first.



We use the AS2303 KV1500 motor throughout the course. Its propeller horn **cannot** be separated from the bell.

3.3.1 What the KV rating means

KV is the motor's *velocity constant*, quoted in rpm per volt. It says how fast the rotor spins per volt applied, **with no load on the shaft**:

$$\omega_0 = k_V\, U \tag{3.1}$$

The constant is not a marketing number - it falls out of the motor's physics. As the rotor turns, its magnets sweep past the windings and induce a voltage that opposes the supply (the *back-EMF*):

$$U_{\text{emf}} = \frac{\omega}{k_V} \tag{3.2}$$

The supply voltage splits between this back-EMF and the ohmic drop across the winding resistance R, so $U = U_{\text{emf}} + R\,I$ and the speed under load becomes

$$\omega = k_V (U - R\,I) \tag{3.3}$$

which is why Equation (3.1) only holds at $I \approx 0$. Draw current and the motor slows down.

A high-KV motor spins fast but gives little torque per amp; a low-KV motor of the same size and mass does the opposite. Both cannot be had at once - the choice is where to sit on that trade-off, and then to recover the missing torque with a reduction.

3.3.2 What this means for our robot

With $k_V = 1500$ rpm/V and a 4S LiPo at its 14.8 V nominal voltage:

$$\omega_0 = 1500 \times 14.8 = 22\,200 \text{ rpm} \qquad k_T = \frac{9.55}{1500} = 6.4 \text{ mN·m/A} \tag{3.4}$$

6.4 mN·m/A is very little torque, too little to drive the wings directly, which is exactly why the 23.04:1 Faulhaber gearbox is there. It multiplies torque by 23.04 and divides speed by the same factor. One revolution of the gearbox output is one full flapping cycle, so

$$f_{\text{flap}} = \frac{\omega}{60\,r} = \frac{22\,200}{60 \times 23.04} = 16.1 \text{ Hz} \tag{3.5}$$

at no load. Under real aerodynamic and hydrodynamic loading the drive tops out near 15 200 rpm, i.e. about **11 Hz** of flapping, the figure that reappears in Module 4.

3.4 Motor shaft

To pair the motor with the gearbox, the stock shaft of the brushless motor is replaced by a custom one. What is left to do is the disassembly and the press fit, which is still a delicate operation: the motor's structure is mostly aluminium and quite thin, for weight savings.

1. Remove the circlip on the shaft with the special pliers. **Don't lose it, nor the brass ring.**
2. Separate the rotor from the stator delicately, by hand.
3. Press the stock shaft out of the rotor bell.

> ⚠ Never load the bell radially
>
> Press the shaft out very carefully, making sure to never apply forces radially on the bell. Use the manual press with appropriate support below the bell to avoid deforming the dome shape. **Do not squeeze the bell with a vice, or even by hand.**

4. Press the new shaft into the bell, making sure not to press on the dome-shaped part of the bell without appropriate support underneath. It is a **tight fit** on the bell and a **clearance fit** through the bearings.

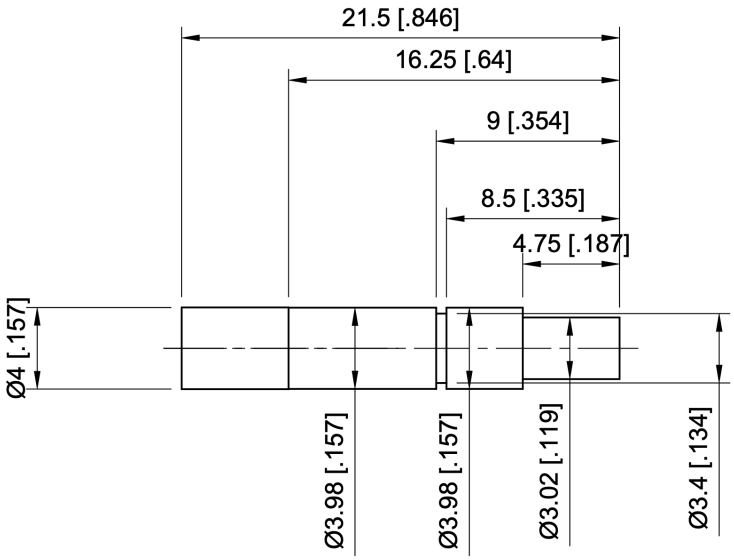


Figure 3.1: Motor shaft dimensions

3.5 Sensor magnet

The AS5047P encoder wired up in Module 3 does not touch the shaft. It reads the direction of the magnetic field from a small magnet rotating in front of it, so the magnet has to go on the motor.

Use a **6 × 2.5 mm diametrically magnetised disc magnet**. *Diametrically* magnetised is the part that matters: the poles are across the diameter, not through the thickness, so the field seen by the sensor sweeps through a full rotation as the shaft turns. An ordinary axially magnetised disc of identical size presents the same pole to the sensor all the way round and gives no angle at all.

1. Drop the magnet into its recess in the 3D-printed bracket. It should sit flat, fully seated, with no rocking.
2. **Centre it on the axis of rotation.** The bracket locates it, but check. Keep the magnet concentric with the shaft to within **0.5 mm**.
3. Bond it with **a single drop of superglue**. One drop is enough.
4. Turn the assembly through a full revolution by hand and watch the magnet. Any visible wobble means it is off-centre or not seated.

> ⚠ Off-centre magnets are hard to diagnose later
>
> An eccentric magnet may still produces a perfectly plausible angle reading at times.

3.6 Preparing the robot bottom plate

Cut a chamfer for every 2mm hole in the bottom plate.

Since we will be using an aluminum fuselage, we will add a paint layer to avoid accidental shorts.

3.7 Mounting the motor

With the shaft and the magnet done, the motor goes onto the robot plate. This happens in two stages: motor to bracket, then bracket to plate.

1. Fix the motor to the vertical **3D-printed nylon bracket** using **four screws M2x5 mm** through the mounting holes in the motor base. Don't over tighten. Check that the rotor still turns freely by hand. Nylon is compliant, and overtightening can pull the bracket out of flat enough to preload the bearings.

A motor that spun freely on the bench and feels stiff once bolted down needs the screws backed off and retightened more gently.

2. Insert square half-nuts in the slots in the motor bracket. Mount the bracket to the **bottom fuselage plate**. Before tightening, check the orientation: the shaft and sun gear must point towards the front of the robot.

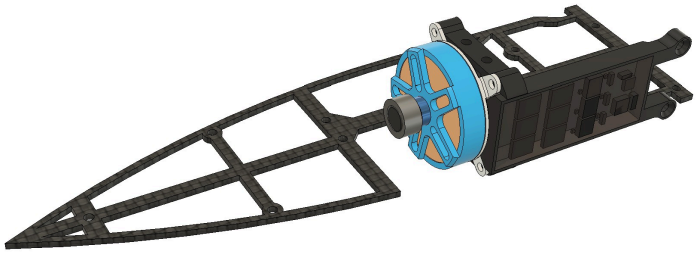


Figure 3.2: Motor and ESC sub-assembly

3.8 Solder the motor wires to the Electronic Speed Controller

Since the pads are large, make sure to use enough heat / large tip on the soldering iron. The order of the cables UVW do not matter.

Watch this before picking up the iron

Soldering Crash Course: Basic Techniques, Tips and Advice

Checkpoint

Shaft seated in motor, the rotor spinning freely by hand with no wobble, the magnet bonded concentric to within 0.5 mm, and the motor bolted to its bracket and mounted on the plate - still turning freely once bolted down. ESC soldered on the motor side.

4. Module 3 — Motor driver, sensor and serial connection

4.1 Summary

We aim to get your laptop talking to a flashed G431 ESC via the flight controller, with the angular sensor wired on SPI. Lots of soldering. We will not use a battery for now, but a cable to a power source instead.

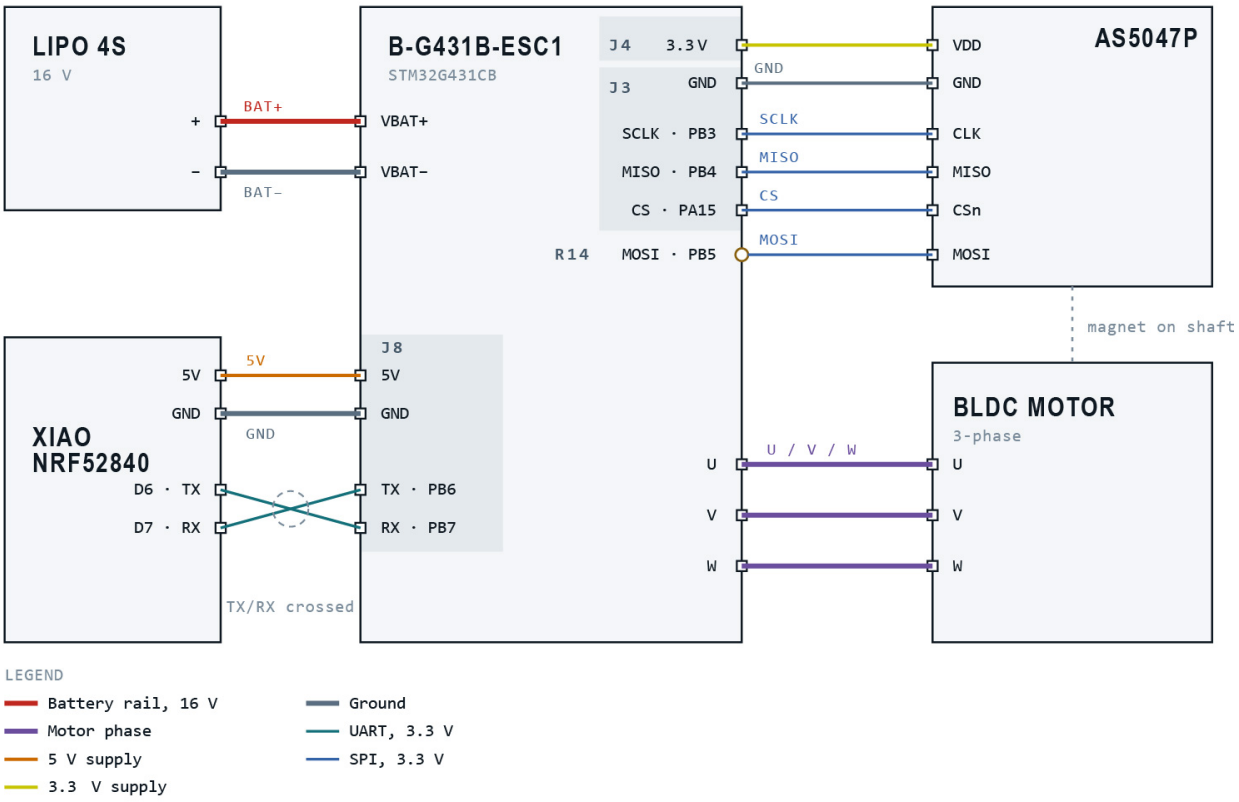


Figure 4.1: How the battery, ESC, encoder, motor and microcontroller connect. The XIAO nRF52840 shown here is the separate-microcontroller option for serial described below - note that TX and RX are crossed. SPI3 and UART pin assignments match the rework in Table 4.2 and *ST-Link serial*.

Work through the module with this diagram open: every step below either wires one of these links or configures it in firmware.

4.2 Requirements

Parts needed	Module 2 sub-assembly, AS5047P encoder, encoder mount, 6x30AWG wire ribbon, power cable.
Tools	Soldering iron, torx drivers, tweezers

Read	This page, plus Design notes on why we use an FOC ESC

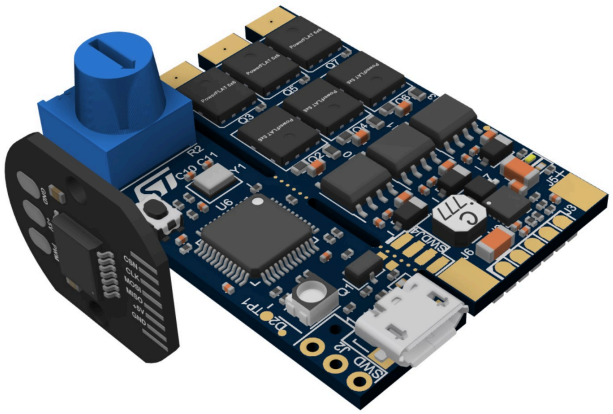


Figure 4.2: ESC running SimpleFOC and the angular sensor. The six MOSFETs are the large flat packages, arranged as three half-bridges — one per motor phase, each with a high-side and a low-side switch. The three copper tabs on the left are the U, V and W outputs the motor is soldered to below.

4.3 Electrical connections

Figure 4.3 is the same set of connections as Figure 4.1, drawn on the actual boards rather than as blocks. Use Figure 4.1 for the pin names, and this one for locating the pad.

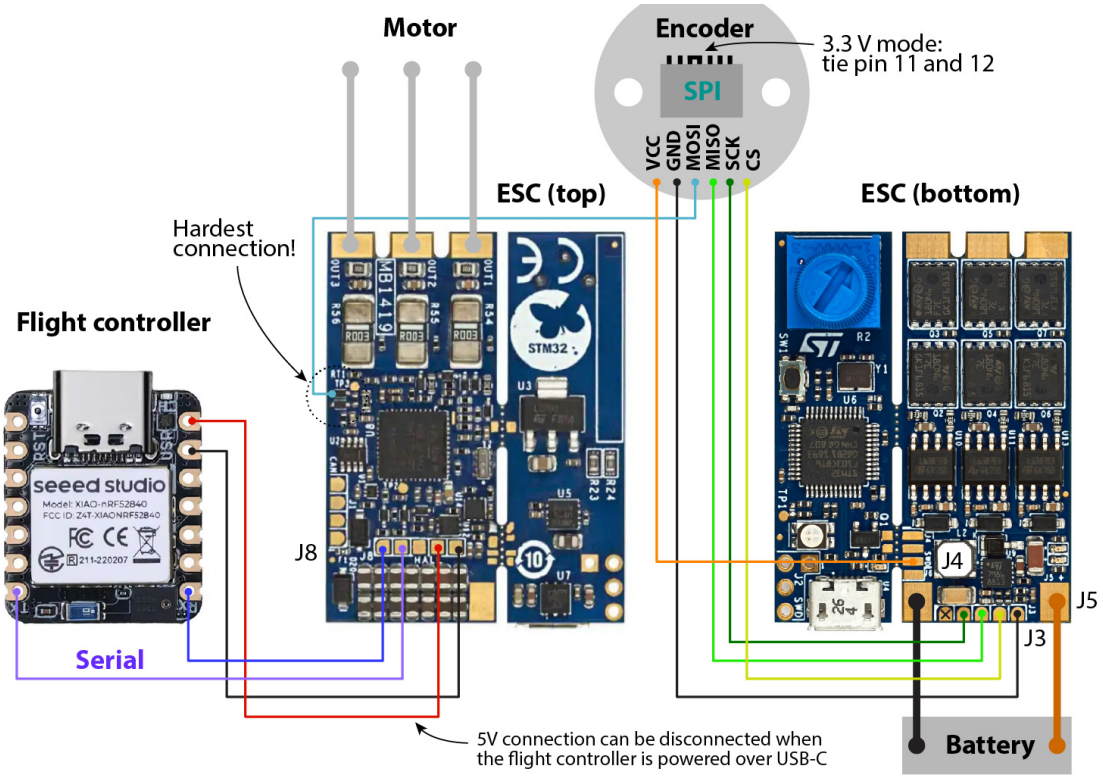


Figure 4.3: The complete electrical system. The 4S battery enters the ESC at **J5**, the three phase wires leave from the large **OUT1–3** tabs to the motor, the AS5047P encoder returns angle over SPI, and the XIAO nRF52840 talks to the ESC over serial. Note that the encoder and the flight controller share the same supply. The **MOSI** tap on **R14**, marked *Hardest connection!*, is the one to leave until last - see Table 4.2.

4.4 Electronic Speed Controller

The **B-G431B-ESC1** is ST's reference three-phase motor driver: a complete ESC on one small board, sold as an evaluation kit. It contains: - **An STM32G431 microcontroller.** A 170 MHz Cortex-M4 with hardware floating point. Motor control recomputes the field orientation every control loop. This MCU is the reason we can. - **A three-phase inverter.** Six power MOSFETs drive the three motor phases. - **A detachable ST-Link programmer (the daughterboard).** The small daughter board is how we program the MCU over USB, and it also carries a serial port back to the laptop. When snapped-off, the MCU cannot be reprogramed without physically resoldering it.

It also brings out many input/outputs (for example**J8**), a CAN transceiver and a potentiometer input, none of which we use as intended. In this build J8 gets repurposed for serial, and the encoder arrives on SPI instead. Both of those reworks are below. The board is rated well above what the wing motor draws (40A), so current is not the constraint here. Get familiar with every component on the board, how six MOSFETs turn one DC rail into three rotating phases, what current it can actually deliver, and why the programmer is a separate snap-off board under The G431 ESC board.

4.5 Flight controller microcontroller

We will use a **Seeeduino XIAO nRF52840 Sense** to send commands to the ESC and record information from the ESC. Why?

- It is small and light. 21 × 17.5 mm and 2.4 g. On a robot that has to move, every gram saved matters.
- It has wireless communication out of the box. The nRF52840 is a Bluetooth Low Energy chip and a microcontroller. Once the robot is sealed in Module 10 BLE becomes the only way commands to interact with the robot during operation, which is exactly how a run is retrieved in Module 11.
- It has somewhere to put the data. 256 kB of RAM and 1 MB of flash on the chip itself are the two logging destinations used during wet testing. No SD card, no extra memory chip, nothing else to waterproof.
- Onboard IMU to log the attitude of the robot without additional components.
- It costs 20$. In case a failure, it can be replaced.

It programs from the Arduino IDE like any other Arduino board, but we highly recommend using **Visual Studio Code + PlatformIO**. The reason for PlatformIO is that it handles multiple-file projects much better, allows git versioning and handles libraries better. It also provides a more performant editor.



Figure 4.4: The Seeeduino XIAO nRF52840. **TX** and **RX** are the two pads at the bottom corners — those are the ones to solder to, and they cross over to the ESC's RX and TX respectively. The board is 21 × 17.5 mm, which is why it can ride on the robot at all.

1. Solder the Seeeduino and the ESC's **Serial1**: TX, RX and **GND** lines. For that, use the 6x30AWG wire ribbon, spliced (only need 4 lines). In Serial communication, TX and RX are **crossed**, each board's transmit line goes to the other's receive line, as drawn in the wiring diagram in Module 3. GND is not optional: without a shared reference the two boards have no agreed voltage for what counts as a high or low bit.
2. Verify that you can command the LED of the seeeduino on and off first. This requires Visual Studio Code and the PlatformIO plugin installed. Leave the ESC unpowered for now.
3. Load the barebone Arduino sketch onto the Seeeduino, modify it if needed, see Software for the repository.
4. Load the default ESC code onto the ESC via its daughterboard (via micro-usb).
5. Prepare the USB-C 15V power source. We are using a USB-C power delivery cable to power the robot on the bench. A PD supply does not simply hand over whatever voltage you want. It advertises a fixed

menu, and the sink has to negotiate for one of them, which is why we use a PD trigger cable or board is needed to pin it at 15 V. Table 4.1 is the power available.

Table 4.1: USB-C Power Delivery fixed-voltage supplies up to 100 W. A source only has to offer the voltages at or below its rating, so a 45 W charger gives 5/9/15 V but not 20 V.

Voltage	Max current	Power	Notes
5 V	3 A	15 W	Default on plug-in, before any negotiation
9 V	3 A	27 W	
15 V	3 A	45 W	**What we use now**
20 V	3 A	60 W	
20 V	5 A	100 W	

6. Solder the 2 power connection to the ESC using 40 cm of 0.5 mm2 silicone cables and connect 15V from a power source. Do not try to run motor for now.
7. Modify the sketch to turn the LED on the **ESC** on and off via commander instructions. We now have confirmed three things:
   a. The Seeeduino can be flashed.
   b. The Seeeduino can be reached over serial.
   c. The ESC is flashed.
   d. The Seeeduino talks with the ESC.

4.6 Encoder (or Angular sensor)

4.6.1 First, why does the motor need an encoder?

Field-Oriented Control works by holding the stator's magnetic field roughly 90° ahead of the rotor's magnets, which is the orientation that produces the most torque per amp. To place the field there, the controller has to know where the rotor is **right now**.

Ordinary drone ESCs infer this from back-EMF, i.e the voltage the spinning rotor induces in the phase that is not currently being driven (Equation (3.2) in Module 2). That voltage is proportional to speed, so as the motor slows the signal sinks into the noise and the estimate falls apart. This causes *loss of synchronisation at low speed* described in Design notes. The AS5047P removes the guess. It measures the field of the magnet bonded to the shaft end and reports the **absolute** angle - valid at standstill just as much as at 15 000 rpm, and correct immediately at power-on with no homing move.

4.6.2 What SPI is, and why we run it at 10 MHz

We will use an AS5047P encoder. The AS5047P reports angle over SPI, a four-wire synchronous bus: the controller drives the clock (**SCLK**), data goes out on **MOSI** and comes back on **MISO**, and **CS** selects the chip. Transfer speed is limited only by how fast the two ends can shift bits. We use SPI because it offers **10 MHz** communication. The reason speed matters is that the angle has to be re-read every single control loop, and a stale angle is bad: the controller ends up placing the stator field where the rotor **was**, not where it is. That error shows up as wasted current and audible noise. The same 10 MHz clock is why the wiring has to stay short, see below.

4.6.3 Wire it to the board

There is no SPI link exposed directly on the ESC board. The solution is to use the **SPI3 bus**, exposed through a few hardware modifications and matching software changes. Figure 4.1 shows the four signals in context, including the MOSI tap on R14.

> ⚠ The micro-USB serial link is lost
>
> Serial2, used by the daughterboard for serial communication will not work anymore as we are using those pins for SPI3 for the encoder instead. Flashing the ESC still works (programming runs over SWD, on different pins) but every command and every value read back now **has to** pass through serial1 (external TX/RX).

The encoder needs power as well as the four bus signals - six wires in total. Replace the default cable and use the 6x30AWG wire ribbon instead. This is small work! Sit down, use a clean soldering iron, take your time and make sure you have lots of light.

Check the R14 connection under the microscope. Use a drop of glue to fix the cable onto the board

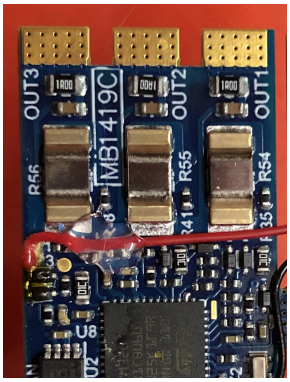


Figure 4.5: The MOSI tap on **R14** under the microscope. The wire lands on the outward side of the resistor, and the bead of clear glue over the joint is to help with accidental pulls on the cable.

Table 4.2: SPI3 connections on the G431. * This connection is on the outward side of the resistor.

	SCLK	MISO	MOSI	CS	VDD	GND
MCU GPIO	PB3	PB4	PB5	PA15	—	—
G431B pin	J3	J3	R14*	J3	J4 (3.3 V)	J3

4.7 Powering the encoder

The encoder is powered by the regulators on the ESC board. Since the STM32 operates at 3.3V, it is best to use the encoder **in 3.3V mode** (it can also work at 5V). The encoder boards may or may not offer the option to run at 3.3V.

The AS5047P chip, however, can work on 3.3V or 5V. For 3.3V mode, the following changes need to be made:

- Remove the capacitor between the 3.3V line and ground (**C2**)
- Connect 3.3V to VDD. For that, tie together pin 11 and 12 on the chip.

> ⚠ Voltage warning
>
> Once these changes made, the marked “5V” connection on the board where the cable is connected should now be connected **to 3.3V only**. Powering it with 5V will destroy the encoder.

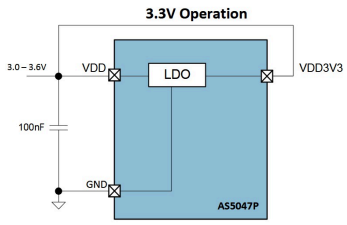


Figure 4.1: AS5047P in 3.3V mode: VDD tied to VDD3V3, bypassing the internal LDO



Figure 4.2: AS5047 breakout board

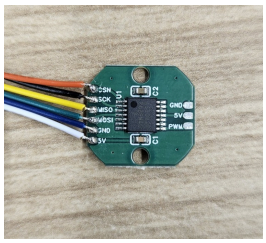


Figure 4.3: Different AS5047 breakout, with the pads labelled CSN, SCK, MISO, MOSI, GND and 5V

Keep the SPI cables short

Cables between encoder and ESC need to be **as short as possible, but still fit in the robot**. At 10 MHz longer runs start corrupting angle readings, and the symptom is a motor that behaves erratically rather than a sensor that obviously fails. Check the length that is needed. The cables also need to be the same length.

4.8 Mounting

1. Install the Seeeduino in its 3D-printed bracket and into the robot
2. Install the encoder in its 3D-printed bracket (the tail stiffener)
3. All mounted to the bottom fuselage plate

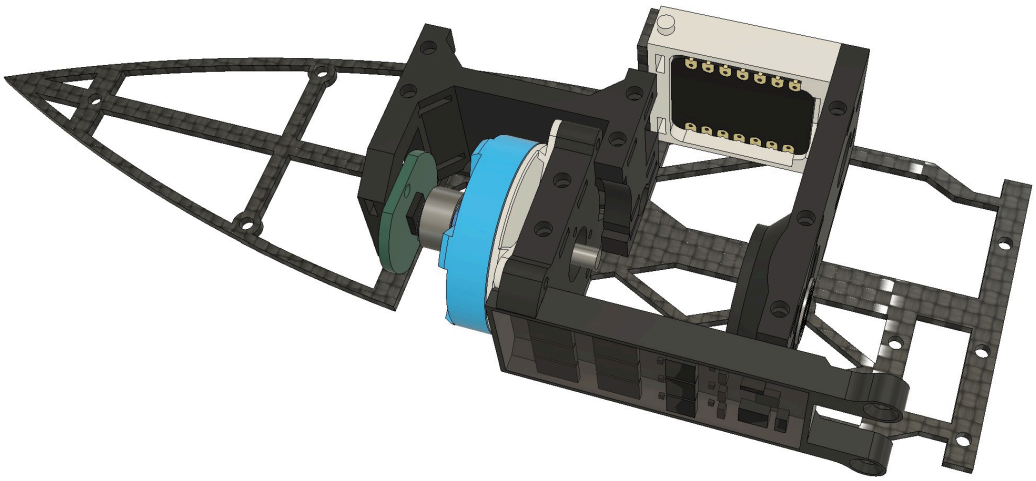


Figure 4.6: Assembled motor-esc

Warning

The first times that you start spinning the motor, it is critical to do that on a **current-limited** power source (**max 1A**), so NOT the USB-C one. Things can go wrong very quickly and if the motor is shorted you only have a few seconds to unplug it.

5. Module 4 — Closed-loop control and tuning

5.1 Summary

The objective of this module is to spin the motor for the first time, understand the parameters at play and optimize gains. The Seeeduino, ESC and encoder are wired already and we will seek to command the ESC via the Seeeduino, running continuously, then holding position, then tracking a velocity setpoint with a tuned loop.

5.2 Requirements

Parts	Motor with new shaft from Module 2, ESC from Module 3, Seeeduino XIAO nRF52840
Tools	Bench supply with current limit, camera, soldering iron
Read	This page in full - the speed limits below explain most of what comes up

5.3 Electronics check

Check the state of the electrical connections to the encoder and to the flight controller. Have someone else verify the connections. Watch out for swapped wires, weak/cold solder connections and check the R14 connection point.

> ⚠ Problems
>
> Most problems with SimpleFOC come from badly soldered cables and miswiring.

5.4 Firmware

The ESC needs to be flashed with SimpleFOC. A custom version of the code for the B-G431B-ESC1 motor controller, using the SimpleFOC library, is to be found here: FAAV-G431-ESC. See Software for all repositories.

Before changing anything in there, read How SimpleFOC works. It covers the two nested loops, what the FOC maths does each iteration, why the motor twitches at power-on, and which settings in the sketch matter - enough to read the firmware rather than guess at it.

5.5 Controlling the motor from a laptop

Next, install **SimpleFOC Studio**: SimpleFOCStudio. It is a desktop GUI that drives the same commander interface described below, so instead of typing commands into a serial monitor you get a form for every motor parameter, live plots of the monitored variables, and a button that dumps the tuned values back out as Arduino code.

Laptops can block the connection to the Seeeduino (over USB-C) if the Seeeduino is powered over 5V through the pin. If that happens, power needs to be disconnected. Two options are possible: either a UBS-C power isolation module or disconnecting the 5V line between ESC and Seeeduino.

> ⚠ Torque control mode
>
> Do not change the torque control mode in the GUI. It will display `voltage mode` but the ESC actually runs a newer `estimated control` mode.

Figure 5.1 shows the parts you will actually use: the parameter tree on the left, the live plot of target, velocity and angle in the middle, and a command line at the bottom for anything the form does not expose.

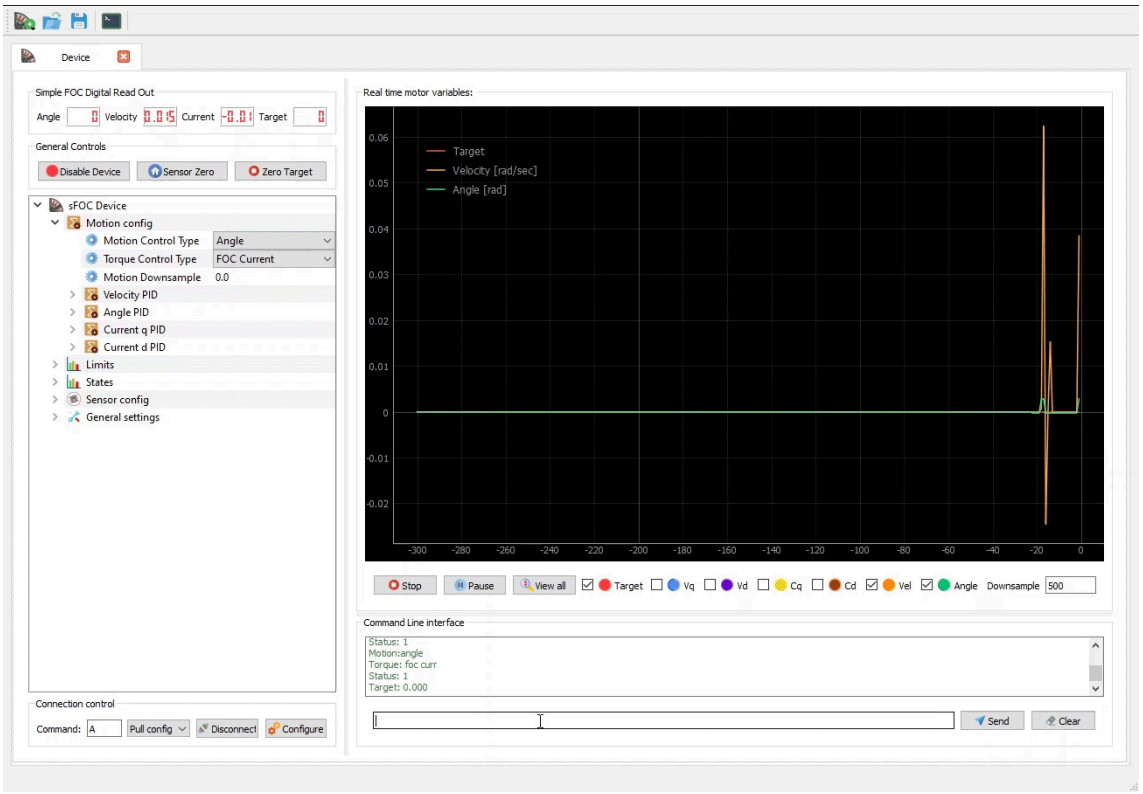


Figure 5.1: SimpleFOC Studio driving a motor. The tree on the left exposes every commander parameter as a form - motion control type, the four PID blocks, limits and sensor config - while the plot follows target, velocity and angle as they change. Screen capture from the SimpleFOCStudio project; it animates on the web and prints as a single frame.

> ℹ What the ESC code has to provide
>
> Studio does not work against arbitrary firmware. The ESC code must register the motor with the commander (`command.add('M', doMotor, "motor")`), enable monitoring with `motor.useMonitoring(Serial)`, and call both `motor.monitor()` and `command.run()` every loop. The command letter set in Studio's device settings has to match the one in the sketch - `M` in our case.
> On this robot the ESC's own micro-USB serial is gone - those pins were repurposed for the encoder's SPI3 bus in Module 3 - so the port Studio opens is the **Seeeduino's** USB serial, and the Seeeduino has to pass what it receives through to the ESC on Serial1 (essentially it needs to act as a relay). Studio cannot reach the ESC on its own.

5.6 Commanding and monitoring the ESC

This is a high-level summary of the SimpleFOC communication docs. The ESC is driven through the **commander**, a practical interface for setting and getting values (instructions). Ask the commander for available commands with `?`. Commands available:

Command	Effect
`M`	Full motor configuration
`F`	Get the main loop frequency
`L`	Turn the onboard LED on and off (useful as a comms check)
`T`	Set the target value - rad, rad/s or N/m depending on control mode
`S`	Get current angle and velocity
`W`	Watchdog on/off. If no new target arrives for 1 s, target is set to zero. **On by default**
`ME`	Enable the driver. `ME1` energises the phases, `ME0` switches them off and the motor coasts
`MC`	Motion control type. `MC0` torque, `MC1` velocity, `MC2` angle
`MLC`	Current limit, in amps

The `M` prefix is the motor command, and the letters after it walk down into its settings - `MC` is the motor's *control* type, `MLC` the motor's *limit* on *current*. That is why full configuration is just `M` on its own.

When it comes to commanding from the flight controlling direction, your code shouldn't write these directly. Instead, you should create wrapper functions such as "`current_limit(value)`" etc...

> ⚠ Careful when changing control mode
>
> `T` means something different in each mode, position, velocity, or torque. Sending a velocity setpoint while in torque mode will do something unintended. BE VERY CAREFUL ABOUT THIS.

5.6.1 Tasks to do:

0. Change the motor command letter (towards the bottom in the interface) to `M`.
1. Commander responds to `?` over serial from a laptop, `L` toggles the onboard LED.
2. The sensor returns a plausible angle through a full manual rotation.

5.7 Tuning the cascaded controller

Figure 5.2 is the structure SimpleFOC implements. We need to tune it.

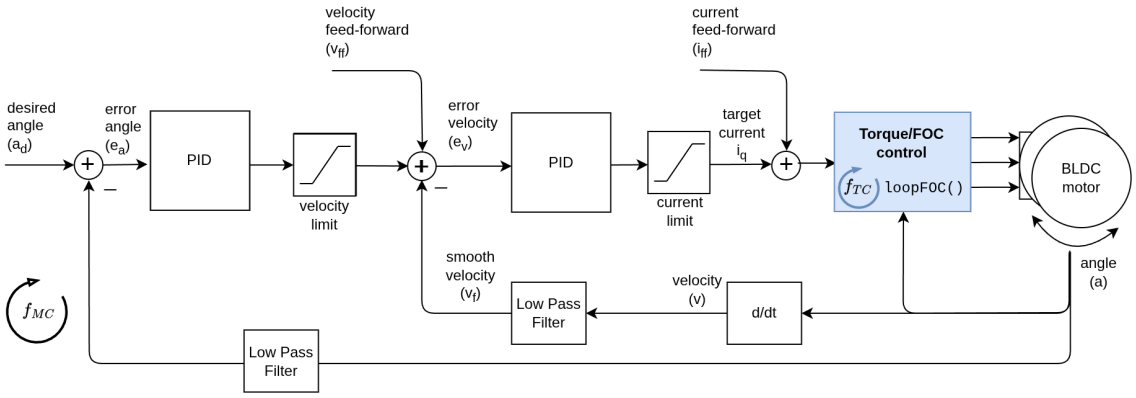


Figure 5.2: The cascaded controller. Desired angle enters on the left. The angle PID does not drive the motor - it produces a *velocity* target, which the velocity limit clamps. The velocity PID turns the velocity error into a *target current*, which the current limit clamps. Only the innermost torque/FOC stage touches the phases. Feedback runs the other way: the sensor returns angle, the derivative gives velocity, and both are low-pass filtered before use. Note the two rates — `loopFOC()` runs the torque stage, `move()` runs the two outer loops, and they are not the same frequency.

Each stage turns its own error into a **setpoint for the stage inside it**. That is the whole idea, and it is why the two clamps in the middle are not decoration: the velocity limit caps what the angle loop is allowed to ask for, and the current limit caps what the velocity loop is allowed to ask for.

Tune from the inside out. Work in this order, and do not move outwards until the loop you are on behaves:

3. **Torque first**, in torque mode (`MC0`). Command a current and check the motor takes it smoothly, without whine or oscillation.
4. **Velocity next** (`MC1`), once torque is solid. Proportional gain first, then integral. Watch the low-pass filter on the velocity feedback while you do: heavy filtering looks like it is cleaning up noise, but the lag it adds is what makes the loop hunt.
5. **Angle last** (`MC2`). Usually proportional gain alone is enough, because the velocity loop underneath is already doing the work.

The order is not a convention, it is a dependency. An outer loop tuned against a sloppy inner loop is partly compensating for the inner loop's error, so its gains encode that error - and they stop being valid the moment you go back and fix the inner loop.

These four PID blocks are exactly what SimpleFOC Studio exposes in its parameter tree (Figure 5.1). That is the practical argument for having installed it: you can change a gain and watch the step response without reflashing between attempts.

SimpleFOC documents each stage separately: torque control, the velocity loop, the angle loop, and cascaded position control for the structure in Figure 5.2 as a whole. The exact forms being tuned are in the PID implementation and the low-pass filter.

5.7.1 Record how it responds to a step change:

Do these in Studio. For each one, capture the step response **and write down the gains that produced it**. A screenshot without its gains is not a result: nobody can reproduce it.

Two things to set before the first capture.

- **Turn the watchdog off** for the duration (`W0`). It is on by default and zeroes the target after 1 s without a new one, so it will pull your step back down in the middle of the capture and look like a controller fault.
- **Plot the variable the loop actually controls** - current in torque mode, velocity in velocity mode, angle in position mode - and leave `Target` enabled in all three so the step is visible.

6. **Torque mode** (`MC0`). Set the velocity limit to 200 rad/s, then **block the output with your hand or a piece of tape before commanding any current** - in torque mode nothing limits speed, so an unblocked motor just accelerates until friction stops it. Set the current limit with `MLC`, starting around 1 A, and step the target.

7. **Velocity mode** (`MC1`). Step from standstill to 400 rad/s and capture the response. Note that this is motor speed, not the flapping frequency of the final system. Mount a small flag (piece of tape) on the output so rotation is visible and countable.
8. **Position mode** (`MC2`). Step from 0 to 144 rad and back to 0 - about a full flap cycle after reduction - and capture the response.

Report rise time, overshoot and any steady-state error read off the plot.

> ⚠ A blocked motor turns every amp into heat
>
> With the output held there is no back-EMF and no mechanical work, so all the electrical power goes into the windings. Apply the torque step in bursts of a few seconds and let the motor cool between attempts if needed.

5.8 Calibration

We'd like to also make sure that we have the right **motor parameters** for running at high speeds (above 400 rad/s). To this, we will follow the commissioning instrutions in the firmware above. Do this with your TA. Run autocal until 1000 rad/s.

5.9 Why we use an ESC that we reprogram

5.9.1 Why low speed is the hard part

Regular drone ESCs start **losing synchronisation at low speed**, which is exactly the regime used for water propulsion. That is the reason this course uses an FOC ESC rather than a hobby unit - see Design notes.

5.9.2 Why high speed is the hard part (going beyond 600 rad/s)

Two issues limit speed: the micro-controller loop rate, and delays from the rotation sensor. Using SPI at 10 MHz and an AS5047P sensor rated for 28kRPM speeds are the required hardware changes that improves on delay. But at very high speeds, sensor lag still becomes the binding constraint, and shows up as high current draw and noise. We need to update the sensor electrical angle offset as a function of speed. Compensating sensor lag as speed increases is necessary to go above 600 rad/s without wasting current. This is automated by ramping through the speeds and minimising phase current. This SimpleFOC forum thread is useful on the topic.

> ℹ Checkpoint
>
> Continuous rotation at 1 Hz, position hold against a hand disturbance, and a recorded loop frequency from `F`. The robot should be tuned, operating in various modes and can be commanded from the flight controller

6. Module 5 — Frame, crank, linkage and gears

6.1 Summary

We assemble the frame, the CNC-cut crank and linkage, the gears, and the full mechanical integration of the 4-bar linkage. Flapping amplitude is fixed by this design to **60 degrees**[1] - 34° up and 26° down, as laid out in Figure 6.2.

6.2 Requirements

Parts	5 mm aluminium link and crank, 2 gears (3D-printed), bearings x5, M2×5 screws, shaft rods, circlip 4 mm, washer 4mm x 0.5mm, M3 screw for crank.
Tools	drill press, metal glue, caliper
Read	This page

6.3 Main body

The main body is a bi-plate structure, spaced **24 mm** apart. The plates are CNC-cut from 1.5 mm plates. The top and bottom plates are identical, except for the chamfers needed for the M2 screws. The screws holding plate to plate are **M2×5**, screwing into square nuts.

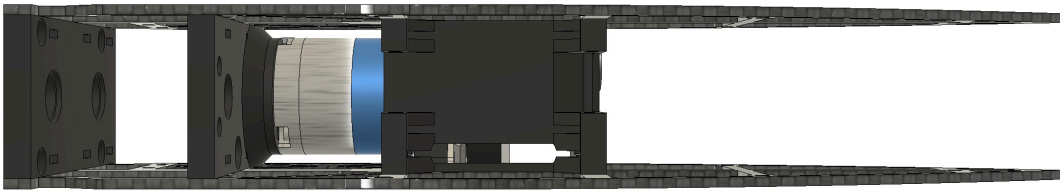


Figure 6.1: Carbon/Aluminum frame structure

6.4 Reduction

The motor will be connected via a custom shaft to a gearbox with a **23.04:1** reduction by Faulhaber[2] that leaves **5 mm of shaft length free** for the crank to be mounted on. A flat on the shaft helps lock the rotation of the M3 screw extending through the crank.

[1] Faulhaber gearhead reduction ratios (accessed 20.02.2024).
[2] Faulhaber gearhead reduction ratios (accessed 20.02.2024).

6.5 Crank and linkage

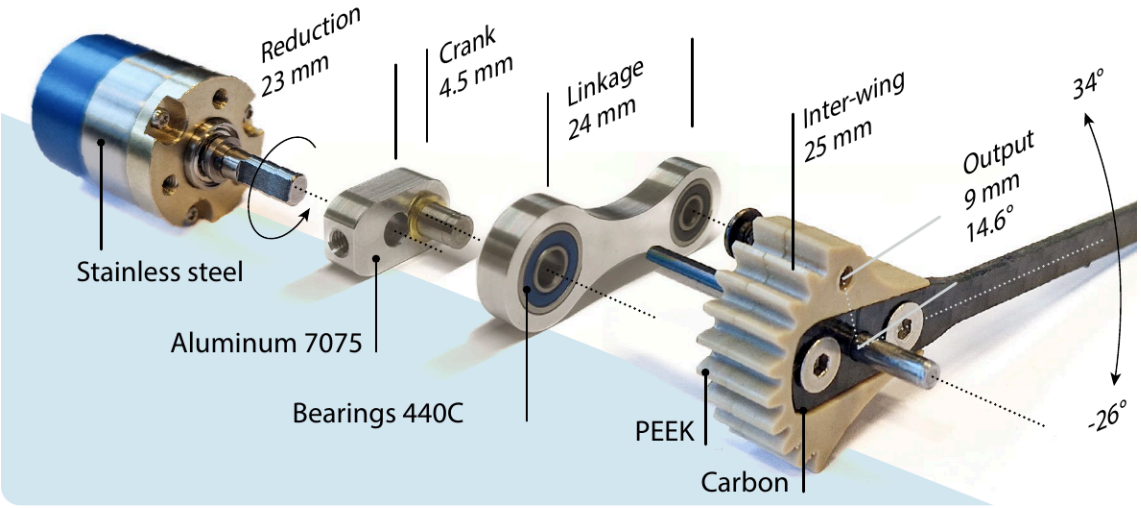


Figure 6.2: The transmission, exploded, with the length that matters and the material for each part. Follow it left to right: the gearbox output turns the crank, the crank drives the linkage, and the linkage swings the inter-wing gear. The **34**° up and **–26**° down at the right are where the 60° flapping amplitude comes from - it is set here, by these lengths, and cannot be changed in software.

The dimensions called out in Figure 6.2 are the ones to cut to:

	Crank-linkage dimensions
Crank radius	4.5 mm
Linkage length	24 mm
Crank–linkage shaft diameter	4 mm
Bearing, in linkage, crank side	10×4×4 mm
Bearing, in linkage, wing side (shaft C)	8×3×4 mm
Bearing, in the middle plate, around Faulhaber reduction	8×4×4 mm flanged
Bearings, around wing gear shafts A and B	7×3×3 mm flanged, 4x

Both the crank and linkage are CNC-cut in AL7075.

6.5.1 Crank and linkage assembly

1. Press the bearings into the linkage. They may need to be bonded with metal glue if the press fit is not good enough.
2. Join the crank and linkage with a short 4 mm shaft (D). The bearing is kept from coming out with a circlip. The CNC-cut shaft already has a groove for a circlip.
   a. Press fit into crank (in the hole that does NOT have the perpendicular M3 thread) until flush with bottom of crank. It needs to be a good press fit, shouldn't move at all.
   b. Then add a **0.5 mm-thick** washer.
   c. Insert the link 10 mm bearing onto the shaft
   d. Lock in place with a 4 mm circlip.
3. Add an M3 screw to mount the crank on the (future) reduction.



Figure 6.3: The parts at the end of this step. The linkage has both bearings pressed in - the 10 mm one at the crank end, the 8 mm one at the wing end - and the crank sits behind it with its 4 mm shaft and the M3 mounting screw alongside.

6.6 Gears and shafts

Cut the shafts to length as indicated below. Select a stainless, 3-mm diameter rod for this.

Shaft	Length
A	15
B	30
C	19

Two of the shafts (A and B) sit at the center of rotation of the wings, and the remaining shaft (C) connects the left gear to the crank link.

The A and B shafts are pressed through the two gears' center holes. Should be a mildly tight fit at minimum, otherwise glue the shaft with a drop of superglue.

Between the gears, a clearance of **0.4 mm** (inter-axis spacing of 25.4 mm) ensures proper operation without too much backlash.

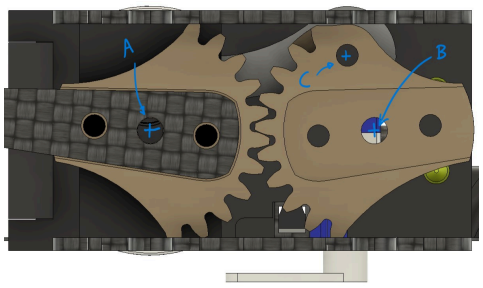


Figure 6.1: Transmission assembled

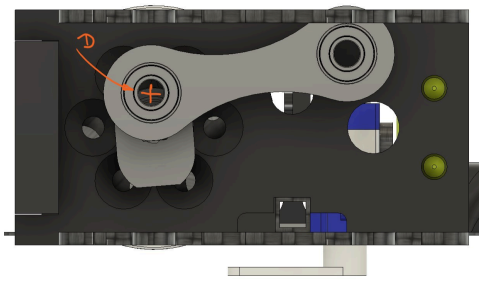


Figure 6.2: Crank-linkage

6.7 Linkage-Gear assembly

Press the last shaft (C) through the right wing gear, with the link in place. In other words, the 3 mm shaft rests on either side of the right gear and keeps the smaller bearing of the link in place. Figure 6.5 is what the module should leave you with. Compare it against your own build. You should now have a right wing gear, connected to the crank and link.

⚠ Alignment of the linkage, perpendicular to the shafts, is extremely important

An offset of **1 mm** is sufficient to destroy the crank-link in a few strokes. Use spacer or even reprint parts if it is not the case.

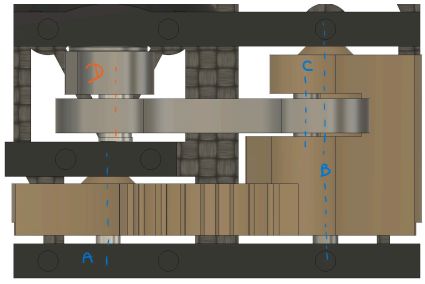


Figure 6.4: Transmission, top view

6.8 Mount the right wing

Insert the flanged bearings into the 3D-printed support structures. Push the right wing + crank-link assembly into place (shaft in bearing). The wing gear has a conical flange that acts as a bearing on the tail side.

6.9 Mount the left wing

Insert the flanged bearings into the 3D-printed support structures. Push the wing into place (shaft in bearing). The wing gear has a conical flange that acts as a bearing on the tail side.

6.10 Insert the front plate

The front plate slides onto the 2 shafts. Lock it in place with the top and bottom screws from the frame.

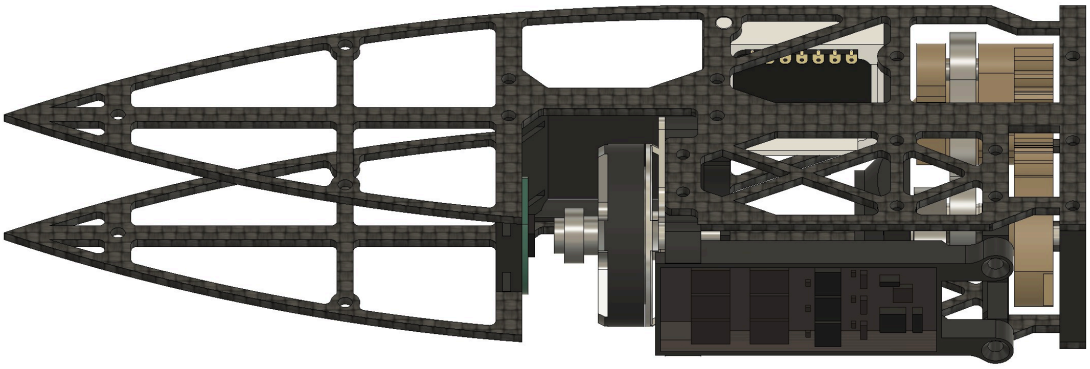


Figure 6.5: The assembled drivetrain in CAD, from above. The carbon frame plates run from the nose at the left; the motor and its gearbox sit on the centreline with the ESC alongside; the crank, linkage and inter-wing gears are at the right. A render rather than a photograph, so use it for part placement and orientation rather than for what the finished hardware looks like.

i Checkpoint

Frame assembled, crank–linkage-gears assembled, linkage visibly perpendicular to the shafts, and gear backlash checked.

7. Module 6 — Flapping simulation (Ptera)

7.1 Summary

The wing and tail geometry are chosen here. Candidate geometries are simulated in Ptera. Modules 7 and 8 build whatever is decided here.

7.2 Requirements

Bring	Laptop with Ptera installed
Read	This page

7.3 What Ptera does

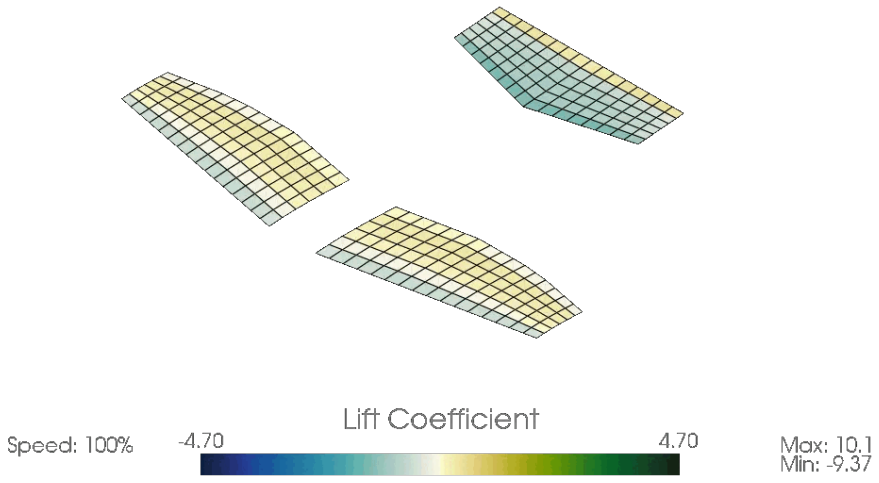


Figure 7.1: A Ptera run: the lifting surfaces are divided into the panel mesh the solver actually works on, and coloured by lift coefficient through the stroke. The deformation you can see is the point - the loading changes continuously as the wing flaps, which is exactly what a steady calculation cannot give you.

Ptera Software is an open-source package for analysing flapping-wing flight. It exists because the usual options are bad: steady aerodynamics assumes the wing is not moving, which is the one assumption a flapping wing breaks, and full CFD is slow and expensive enough that you cannot use it to compare candidate planforms in an afternoon.

It uses the **unsteady ring vortex lattice method (UVLM)**. The surfaces are reduced to a mesh of panels, each carrying a vortex ring, and the wake is shed and tracked as the wing moves. That is a large step up from a steady method - it captures the history of the flow, so the forces at one instant depend on what the wing did before - while staying fast enough to sweep a design space.

7.4 Installing Ptera

The examples are in the repository, so clone it rather than installing the package on its own:

```
git clone https://github.com/camUrban/PteraSoftware.git
cd PteraSoftware
pip install -r requirements.txt
```

Then run the unsteady example to confirm the install works:

```
python examples/unsteady_ring_vortex_lattice_method_solver_variable.py
```

It should open an animation like Figure 7.1. If it does, the solver, the geometry classes and the output stack are all working and you can start editing a copy of that script into your own geometry.

> **i** Two ways to install it
>
> `pip install pterasoftware` also works if you only want the library. It does **not** give you the `examples/` folder, which is where the working scripts are.

7.5 How a Ptera model is put together

The thing worth understanding before you open the example is that **geometry and motion are two parallel hierarchies**. You build the shape first, then build a matching movement object for each piece of that shape. Every geometry class has a movement twin.

Geometry	Movement twin	What it is
`Airfoil`	—	The 2D section. Does not move on its own
`WingCrossSection`	`WingCrossSectionMovement`	An airfoil placed at a span position, with its own chord, twist and offset
`Wing`	`WingMovement`	Two or more cross-sections swept into a surface
`Airplane`	`AirplaneMovement`	All the wings together — on this robot, the two wings and the tail
`OperatingPoint`	`OperatingPointMovement`	Freestream velocity, density and angles

The two sides are then joined and solved:

1. `Movement` takes the `AirplaneMovement` and the `OperatingPointMovement` and becomes the complete time-varying description of the problem.
2. `UnsteadyProblem` wraps that `Movement`.
3. `UnsteadyRingVortexLatticeMethodSolver` takes the problem and runs it.

So the flapping is **not** a property of the wing. It lives in the movement twin, which is why the same geometry can be swept through different kinematics without rebuilding it.

7.6 Modelling the tail

In Ptera it is simply another `Wing` on the `Airplane`, built the same way as the flapping wings: two or more `WingCrossSection` objects, each with an `Airfoil`, swept into a surface. It needs a `WingMovement` like everything else, but for a fixed tail that movement is static (the object still has to exist).

The planform follows from where you put the cross-sections and what chord you give each one:

- **Rectangular** - same chord at root and tip. The simplest thing to build and the easiest baseline to compare against.
- **Tapered** - smaller chord at the tip. Less area outboard, less inertia at the end of the moment arm.
- **Delta or triangular** - taper taken to the limit, chord going to near zero at the tip.

- **Forked** - two surfaces leaving a common root, the shape many fish converge on.

7.7 Beyond prescribed motion

Everything above prescribes the motion completely: you say where the wing is at every instant, and the solver reports the loads. Ptera has two beta features that go beyond that

Free flight hands the airplane to the MuJoCo physics engine, so it flies a six-degree-of-freedom path under its own aerodynamic loads, weight and inertia: the question of whether a chosen geometry actually sustains flight or what maximum speed it can reach in swimming, rather than what lift it makes while held in place.

Aeroelasticity lets each wing twist under load through a torsional spring at every cross-section, which matters here because a membrane wing is not rigid and the twist changes the loading.

Both are switched on the same way: the geometry stays exactly as you built it, and you swap the *movement, problem and solver* classes for a different family. The naming follows one rule - take the standard class and put `FreeFlight` or `Aeroelastic` in front of it, so the free-flight solver is `FreeFlightUnsteadyRingVortexLatticeMethodSolver`.

What separates the two families is *which* classes take the prefix.

Table 7.1: Which classes change in each family. The geometry classes - `Airfoil`, `WingCrossSection`, `Wing` and `Airplane` - are identical in all three.

Standard class	Free flight	Aeroelastic
`WingCrossSectionMovement`	unchanged	prefixed
`WingMovement`	unchanged	prefixed
`AirplaneMovement`	unchanged	prefixed
`OperatingPointMovement`	prefixed	unchanged
`Movement`	prefixed	prefixed
`UnsteadyProblem`	prefixed	prefixed
`UnsteadyRingVortexLatticeMethodSolver`	prefixed	prefixed

7.7.1 Free flight

In a standard run you prescribe the operating point at every step. In free flight you give only the **first** one. After that MuJoCo integrates the rigid-body equations and the solver rebuilds each new operating point - speed, angle of attack, sideslip, attitude, position, body rates - from the result.

MuJoCo treats the airplane as one rigid body on a free joint, with the mass and inertia matrix you supply. Ptera applies the entire load itself, the UVLM forces and moments plus anything from `external_loads_fn`, rotated into Earth axes, plus the weight.

A run has two phases. During the **prescribed phase** the loads are withheld and the body coasts at its initial condition while the wake develops, which keeps the impulsive-start transient out of the dynamics. During the **free phase** the loads are applied and the airplane flies on its own. Wing motion stays prescribed throughout.

> **i** Why a free step costs several times a prescribed one
>
> The load over a step depends on where the body ends the step, and where it ends depends on the load. Using the previous step's load is the obvious approach and it **diverges** for flapping cases, because of added mass: a velocity change alters the circulation, which alters the load, which alters the velocity again, with a loop gain of roughly added mass over vehicle mass.
>
> Each free step therefore solves a small fixed-point problem instead, iterating on the twelve-number body state until the weighted residual falls below a relative tolerance of `1e-6`, or until `k_max` iterations. Every pass costs one UVLM solve *and* one MuJoCo step. The full derivation is in `docs/STRONG_COUPLING.md`.

Free flight is enabled by building the free-flight class family instead of the standard one. The script below is a trimmed-down version of `examples/free_flight_unsteady_ring_vortex_lattice_method_solver_glider.py`.

1. **Build the `Airplane`** as usual. Its `weight` must equal `mass * |g_E|`, or the problem constructor raises a `ValueError`.
2. **Wrap each `Wing` in a `WingMovement`** and the airplane in an `AirplaneMovement`. Leave the amplitudes at zero for a glider, or set them to flap. Loop over `airplane.wings`, not over the wings you typed: a symmetric wing whose symmetry plane is offset is split into two `Wing` objects, and each needs its own movement.
3. **Define the initial `OperatingPoint`.** This is the only operating point you prescribe. Set `g_E` explicitly, because the default is no gravity. Keep `externalFX_W=0.0`.
4. **Wrap it in a `FreeFlightOperatingPointMovement`**, then build a `FreeFlightMovement` with exactly one airplane movement, the time step, and the two phase lengths.
5. **Build the `FreeFlightUnsteadyProblem`** with the mass (kg) and the inertia matrix (kg m^2, in body axes, about the CG).
6. **Run the `FreeFlightUnsteadyRingVortexLatticeMethodSolver`**, then use the usual output functions.

Parameter	Default	What to know
`g_E` (OperatingPoint)	zero	**No gravity unless you set it.** Earth z points down, so standard gravity is (0, 0, 9.80665)
`weight` (Airplane), `mass` (problem)	none	Must satisfy weight = mass × gravity, or the constructor raises. For zero gravity set both to zero
`I_BP1_CgP1` (problem)	none	3×3 symmetric, kg·m², about the CG, in **body** axes (forward, right, down). Geometry axes point aft, right, up — if CAD gives it in geometry axes, flip the sign of Ixy and Iyz
`externalFX_W` (OperatingPoint)	0.0	Must stay zero; free flight raises otherwise. Use `external_loads_fn` for thrust
`external_loads_fn` (problem)	none	Returns force and moment as two 3-vectors in **wind** axes. Wind +x points along the velocity, so thrust along the flight path is (T, 0, 0)
`prescribed_num_steps`, `free_num_steps`	none	Both at least 1. The examples coast for 5 to 10 steps
`delta_time`	none	Rule of thumb: the airplane travels about one chordwise panel length per step
`k_max` (problem)	20	Cap on coupling iterations per step. Raise it if the log reports reaching the cap without converging - light vehicles need more
`integrator` (problem)	`"RK4"`	Prefer an implicit one once you add contact geometry
`CgP1_E_Eo`, `omegas_BP1__E`	zeros	Initial position and body rates. Only the free-flight solver accepts nonzero body rates

7.7.2 Aeroelasticity

The structural model is deliberately first-order. Each spanwise **strip** (chord element, i.e. the panels between two neighbouring cross-sections) is a rigid plate hinged **at its leading edge** on a torsional spring, free only to twist about its spanwise axis. There is no spanwise bending in the current model, strips are not connected to each other structurally, and the root is clamped.

Each strip's twist angle follows a forced, damped oscillator,

$$I\,\ddot{\theta} + c\,\dot{\theta} + k\,\theta = M_{\text{aero}} + M_{\text{inertial}} \tag{7.1}$$

with the terms in Equation (7.1) coming from:

Symbol	Meaning	Where it comes from
θ	Strip twist (rad)	Solved for
k	Torsional stiffness (N·m/rad)	`spring_constant_rad`
c	Torsional damping (N·m·s/rad)	`damping_constant_rad`
I	Strip torsional inertia, ½ML²	M = `wing_density` × strip area, L = mean strip chord
M_{aero}	Sum of the strip's panel pitching moments about its leading edge	UVLM loads at the current step
M_{inertial}	Moment from the prescribed flapping acceleration	Flapping inertia × flap angular acceleration

Each step runs once through the loop: the UVLM solves on the current, already deformed geometry; the solver sums each strip's panel moments about its leading edge; SciPy integrates Equation (7.1) over one time step from that strip's previous twist and twist rate; and the new twist is added to the outboard cross-section's angle before the next step is meshed. That is **loose coupling**: the twist at one step uses the loads from the last one, with no sub-iteration. For the first few steps (`step_discards`, default 5) the twist is held at zero, because the impulsive start produces large unphysical loads. Only wings built with an `AeroelasticWingMovement` deform; a plain `WingMovement` in the same airplane stays rigid (which is how you model a flexible wing against a rigid tail).

To use aeroelasticity:

1. **Mesh the deforming wing as thin strips.** The spring model only behaves when each strip is narrow. Set `explode_into_strips=True` on the `Wing`, which requires uniform spanwise spacing on every non-tip cross-section.
2. **Offset the wing root from the symmetry plane.** A wing lying on the symmetry plane raises an error; offsetting it splits the wing into a left and a right `Wing`, which is what the solver expects.
3. **Flap with sine spacing.** The inertial moment needs the flap *acceleration*, so sawtooth spacing is rejected.
4. **Wrap all wing movements in an `AeroelasticAirplaneMovement`.** Rigid wings can keep a plain `WingMovement`.
5. **Use an ordinary `OperatingPoint` and `OperatingPointMovement`**, then build the `AeroelasticMovement` and the `AeroelasticUnsteadyProblem` with the structural parameters.
6. **Run the `AeroelasticUnsteadyRingVortexLatticeMethodSolver`** and read the twist history from the problem.

Parameter	Default	What to know
`spring_constant_rad`	none	Stiffness in N·m/rad. Static twist is roughly moment over stiffness, so a softer spring twists more. Examples use 1000. Too soft distorts the mesh
`damping_constant_rad`	none	Damping in N·m·s/rad. Examples use 1000, which is heavily damped - lower it to see the strips oscillate
`wing_density`	none	Mass per unit wing area, kg/m². Sets both inertias. **Zero breaks the equation**

Parameter	Default	What to know
`step_discards`	5	Start-up steps with twist held at zero. Raise it if the first deformed steps look violent
`explode_into_strips` (Wing)	False	Set true on deforming wings
`prescribed_wake` (run)	True	True is much faster and fine for parameter sweeps; false (free wake) is more accurate

! These two features are experimental

Both features first shipped in **v5.1.0** (July 2026), which is what `pip install pterasoftware` gives you. The behaviour described here follows the newer `main` branch, which differs in several ways: class names are flat (`ps.FreeFlightMovement`) rather than long module paths, a free-flight run starts at the operating point's initial position rather than always at the origin

i Checkpoint

A geometry chosen, with simulation evidence, and confirmed buildable against the frame and wing-root hardware.

8. Module 7 — Wing fabrication

8.1 Summary

Both wings: carbon skeleton, membrane cut to the geometry chosen in Module 6, bonded and stitched at the load-bearing points.

8.2 Requirements

Parts	Carbon rods and strips, wing roots, 2× nylon adapters per wing, ripstop nylon membrane, 3M tape, superglue, thread
Tools	Cutting mat, steel rule, scalpel, needle, printed full-size template of the chosen geometry

8.3 Wing roots

The wing roots are carbon, cut to mate with the gears from Module 5. A champfer needs to be cut in the wing roots for the countersunk head of the M3 screw.

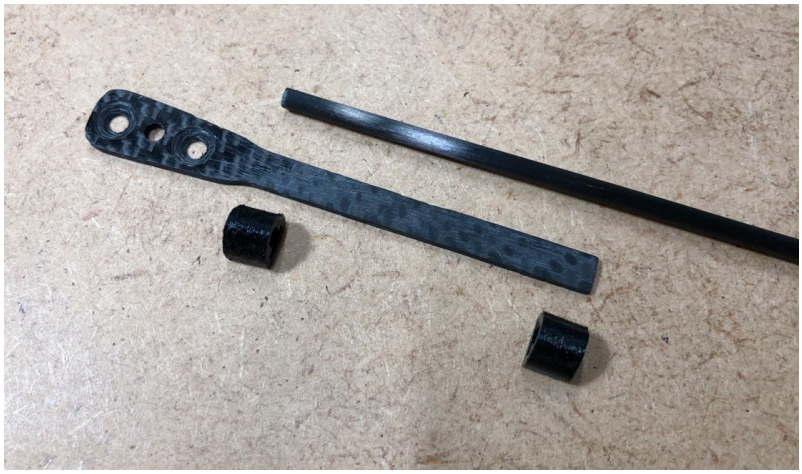


Figure 8.1: Wing root parts and spar stock

8.4 Example and tips for building the wing

The whole wing is built **over the printed pattern**, working from the leading edge outwards. Everything is cut oversize and trimmed to the outline at the very end - margin is easy to remove and impossible to add back.

1. **Decide on the shape** and print the pattern on paper. Print it **1:1** - check the scale before cutting anything, by measuring a known dimension on the printout against a rule. Printer drivers scale to fit by default, and a pattern that came out at 97% quietly discards the geometry chosen in Module 6.
2. **Cut two membranes** with a **3 cm margin** all round the pattern. Use the ripstop nylon listed in Parts.

3. **Bond the leading edge.** Lay a strip of **3M tape** along the leading-edge region of the membrane, press the leading-edge rod down onto the tape, and wrap the membrane around the rod. This joint carries the highest load of any on the wing — everything downstream pulls against it.
4. **Lay out the carbon strips.** Cut each to approximate length **plus 3 cm**, position them on the membrane over the pattern, and bond them with **superglue**.
5. **Stitch the strips down**, at minimum at every **intersection point** and at the **extremities**. Superglue alone will peel off the membrane under repeated flapping; the stitches are what keep the skeleton attached once the glue starts to fail.
6. **Trim the excess** beyond the outline with a blade, cutting on the mat.

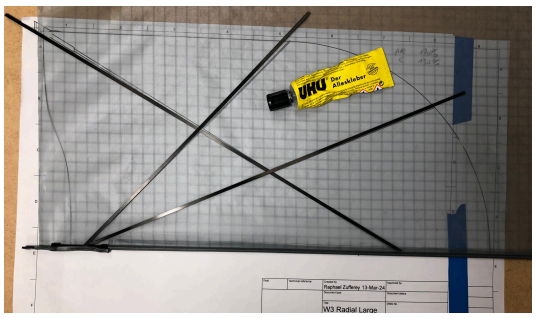


Figure 8.1: Strips laid out and bonded over the template

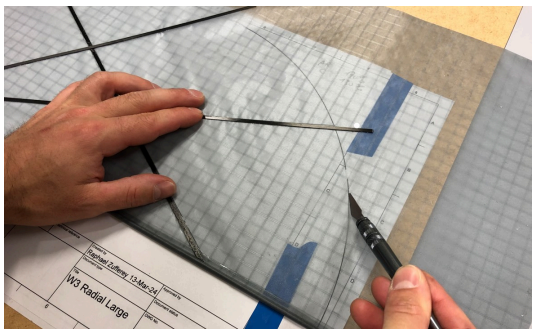


Figure 8.2: Trimming the membrane to the outline

Repeat for the second wing, and remember that it is flipped.

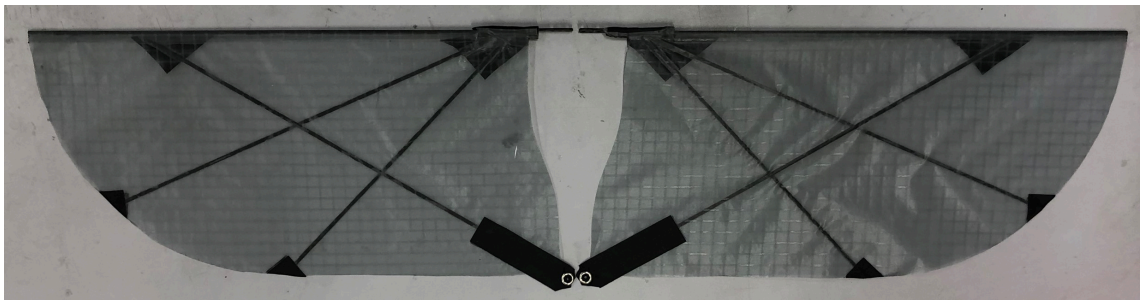


Figure 8.2: Completed wing pair

8.5 Assembly

1. Mount each wing in its **wing root** using the **two nylon adapters**.
2. Mount the wing roots in the robot, onto the gears from Module 5.

> ⚠ Both wings must match
>
> Asymmetry between left and right shows up as a roll bias that may not be possible to correct later. See Module 11 for trimming out. Weigh both wings and compare spar positions against the template before bonding.

9. Module 8 — Tail

9.1 Summary

The passive tail, cut to the shape the simulation produced and mounted on its rod and the trailing edge mount ## Requirements

Parts	Carbon sheet and tail rod, foam stock (XPS or closed-cell PE), epoxy
Tools	Foam cutter, scalpel
Bring	The Module 6 tail geometry, printed 1:1

9.2 Reduction installation

The motor is connected via a custom shaft to a gearbox with a **23.04:1** reduction by Faulhaber.[3] Both parts are held in place by 3D-printed plates. The output shaft of the gearbox has its own bearing on the inside of the gearbox, and is also supported mid-way on the shaft by a 4×8 mm flanged bearing. This leaves **5 mm of shaft length free** for the crank to be mounted on. A flat on the shaft helps lock the rotation of the M3 screw extending through the crank.



Figure 9.1: Faulhaber gearbox

First, we need to press + glue our sun gear is already pressed onto the motor shaft from Module 2.
Next, we need to install the gearbox to the motor and fixing the pair to the plates.

1. Slide the gearbox onto the motor shaft so the sun gear engages the planets. It should seat with light hand pressure, if it resists, back off and check the sun gear is square on the shaft rather than forcing it.
2. Fix the blue part of the gearbox: the custom screws that come with the gearbox on one side, and **M2.5×10** screws on the other.

[3] Faulhaber gearhead reduction ratios (accessed 20.02.2024).

3. Check the output turns freely by hand through a full revolution before applying power.
4. Close the gearbox: the **silver part threads onto the blue part**. Don't tighten it.
5. Attach the **second vertical bracket** to the silver part of the gearbox with **3 × M2.5** screws.
6. Secure that bracket to the **bottom fuselage plate** with **3 × M2** screws. The motor bracket from Module 2 is already on the same plate, so between them the two brackets set the alignment of the whole drivetrain. Check the output still turns freely once both are bolted down.

9.2.1 Sun gear

The sun gear from the Faulhaber 23.04:1 gearbox. Glue and press onto shaft. Check that it fits into the reduction, without touching the first stage. Use a caliper for this.

9.3 Tail

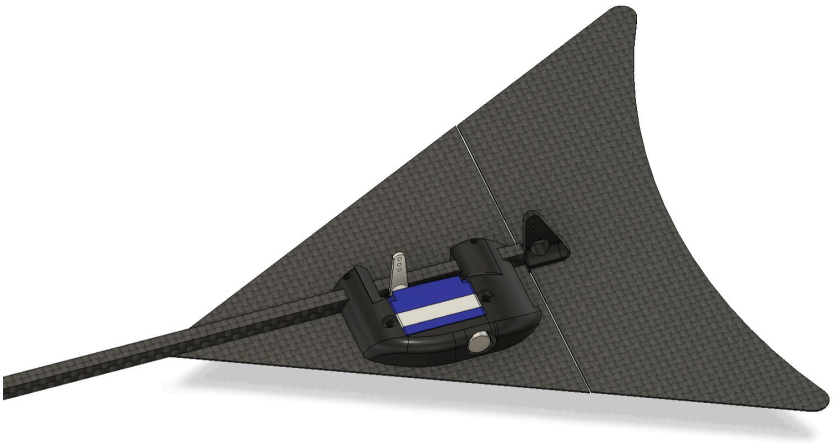


Figure 9.1: Tail assembly

> ! TBC
>
> As with the wings, the guide contains **only the photograph above** for the tail. Still needed: sheet material and thickness, rod diameter, how the tail mounts to the boom, and the incidence setting. The project plan also notes tail actuation is optional if time permits — decide before the semester whether that is in scope.

9.4 Trailing edge holder

The wing root is driven by the gears and sweeps through the full **60°** set in Module 5. The trailing edge is not driven. It is anchored to the **tail tube**, and that anchor is what makes the membrane twist as the wing sweeps. It sets the angle of attack the wing sees through the stroke. Move it and the wing behaves differently. So this part is a trim adjustment as much as it is a mount, which is why both its positions have to stay changeable after the robot is built.

This part is a design exercise. The requirements are fixed; the mechanism is not.

Table 9.1: What the trailing edge holder has to do

Requirement	Why
Rotates with the wing through the whole stroke	The root swings 34° up and 26° down. An anchor that cannot follow loads the membrane in the one direction it cannot take.
Adjustable **longitudinally**, along the tube	Fore/aft position sets wing twist. This is the trim actually reached for in Module 11.
Each Wing mounts can change **laterally and longitudinally**	Sets how far the anchor stands off the boom, and is how left and right are matched.
Wing comes off without dismantling anything	Wings get swapped, damaged and rebuilt. Nothing about this joint may be permanent.
Survives repeated wet cycling	To not fail at competition

9.4.1 Find a solution

Both flexible and rigid solutions are possible. For a rigid solution, the mount half is straightforward: a **split collar** that slides along the 5 mm tube and clamps with an M2 screw into a square half nut, per the house convention in Design notes. Sliding it gives the longitudinal adjustment, rotating it about the tube gives some of the lateral, and a slotted arm gives the rest.

The design must be justified against Table 9.1, and the choices defended.

9.4.2 Print it

1. 3D-print the part with the material required.

9.4.3 Insert it on the tube

Slide the collar onto the tail tube and set it to the starting station. Measure from a fixed datum, write the number down, and set the second one to the same number.

9.4.4 Attach the wing

Connect each wing's trailing edge to its holder.

> ⚠ Both sides must be set identically
>
> Different longitudinal stations left and right means different twist left and right, which is a roll bias -the same failure mode as mismatched wings in Module 7, arriving by a different route and just as hard to trim out in Module 11.

> ℹ Checkpoint
>
> Tail mounted, wings attached to tail.

10. Module 9 — Flight controller, PCB and dry testing

10.1 Summary

The custom PCB assembled and validated, the flight controller flashed and talking to the ESC, and the robot flapping in air under wireless control.

10.2 Requirements

Parts	Seeeduino XIAO, connectors, battery
Tools	Hotplate, tweezers, multimeter, bench supply with current limit
Read	This page

10.3 Flight controller



Figure 10.1: Microcontroller in its waterproofing box

Firmware lives in FAAV-Flight-Controller

10.4 Connect battery

Add instructions

10.5 Dry testing

! TBC

The dry-test protocol is course material, not in the guide. Needed: what the robot is mounted on while flapping in air, the flap frequency sweep, what is measured, the wireless control setup students reprogram to, and abort criteria.

> **Checkpoint**
>
> Flight controller commanding the ESC over serial, and a supervised flapping run in air.

11. Module 10 — Waterproofing

11.1 Summary

A potted PCB, a lubricated and sealed gearbox, and a robot that survives a static immersion test.

> ⚠ Silicone cure time gates this module
>
> The PCB potting takes **12–24 hours** to cure because of the depth. Plan the module so potting happens first and soldering happens after the cure, or across two sittings.

11.2 Requirements

Parts	Waterproofing box, Kapton tape, silicone sealant, USB-C cable, CorrosionX, white lithium grease
Tools	Needle, soldering iron
Read	This whole page before starting — the ordering matters

11.3 Flight controller waterproofing

First ensure that the FC fits within the 3D box before anything else.

1. Fill the bottom of the box with silicone sealant until it reaches the USB port hole.
2. *Carefully* slide the assembly in, with the Seeeduino facing the bottom of the box.
3. **Very important!** Insert a USB-C cable into the port. This keeps the sealant from flowing into the port, and the cable is a convenient point to hold the assembly by.
4. Add sealant along the three free edges to fill any gaps, until the sealant reaches the level of the top side of the PCB.
5. Let the silicone cure - because of the depth this takes about 12–24 h.

11.4 ESC waterproofing

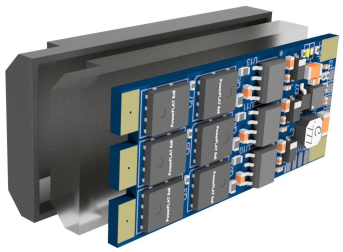


Figure 11.1: The B-G431B-ESC1 assembly.

The ESC is potted the same way as the flight controller: it sits in its own 3D-printed frame, which is then filled with **Silcoset** so only the wire exits remain open. Figure 11.2 shows the three pieces together - motor, bare ESC, and the potted frame.

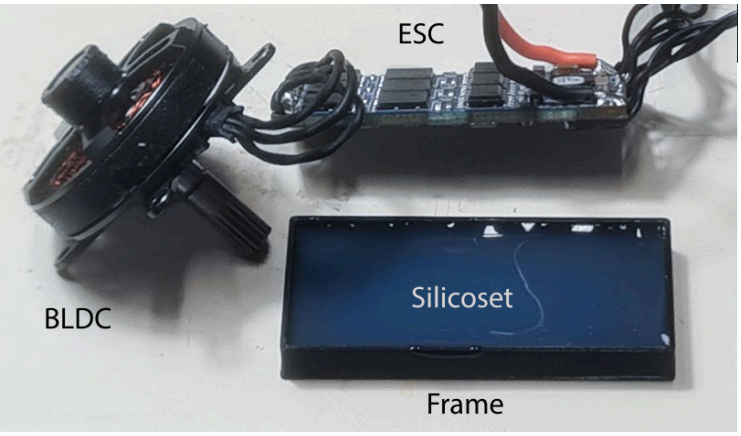


Figure 11.2: The motor, the ESC and the potted frame. The dark block is the frame filled with Silcoset; the ESC drops into it before the fill so the board ends up fully encapsulated with only the phase and signal wires leaving.

11.5 Battery waterproofing

TBC

11.6 Battery connector waterproofing

TBC

11.7 Spend time discussing COT and measurement

TBC

11.8 Gearbox

By default the gearbox on the FAAV4 robot is **not waterproof at all**, and it is normal for it to end up completely filled with water once submerged. Over years of use the gearbox proved resistant to fresh water with the factory lubricant. If that is removed/cleaned, then use the right lubricant below. The best is to avoid opening the gearbox and cleaning the gears.

> ⚠ Use the right lubricant
>
> A mix of **CorrosionX and white lithium grease**. Several dives have since been performed with the same gearbox that previously corroded, and it now runs smoothly.

Parts that rust: **sun gear, planet gears, ring gear, planet shafts.** Parts that do not: screws, motor shaft, washers, body, circlip.

> ℹ Checkpoint
>
> Flight controller and ESC fully cured and potted, static immersion test passed with no ingress. Robot cleaned with IPA after each immersion.

12. Module 11 — Wet testing

12.1 Summary

Immersion, underwater runs, wing and tail trim, and a logged cost-of-transport measurement.

12.2 Requirements

Bring	Fully waterproofed robot from Module 10, charged 4S pack, spare crank–linkage shaft
Tools	Retrieval poles and towels are at the tank — see Tank rules
Read	This page, and the notes from Module 4

12.3 Pre-immersion checks

- ☐ Static immersion test from Module 10 passed
- ☐ Connector closed. USB-C port sealed
- ☐ Watchdog enabled on the ESC (`W`) - target drops to zero if commands stop
- ☐ Robot floats level. If not, add floats.

12.4 The tank

Testing happens in the AURA Lab tank. The gantry along the near rail and the cameras overhead are part of the measurement setup, not scenery — the ceiling cameras are what track the robot for the cost-of-transport number below.

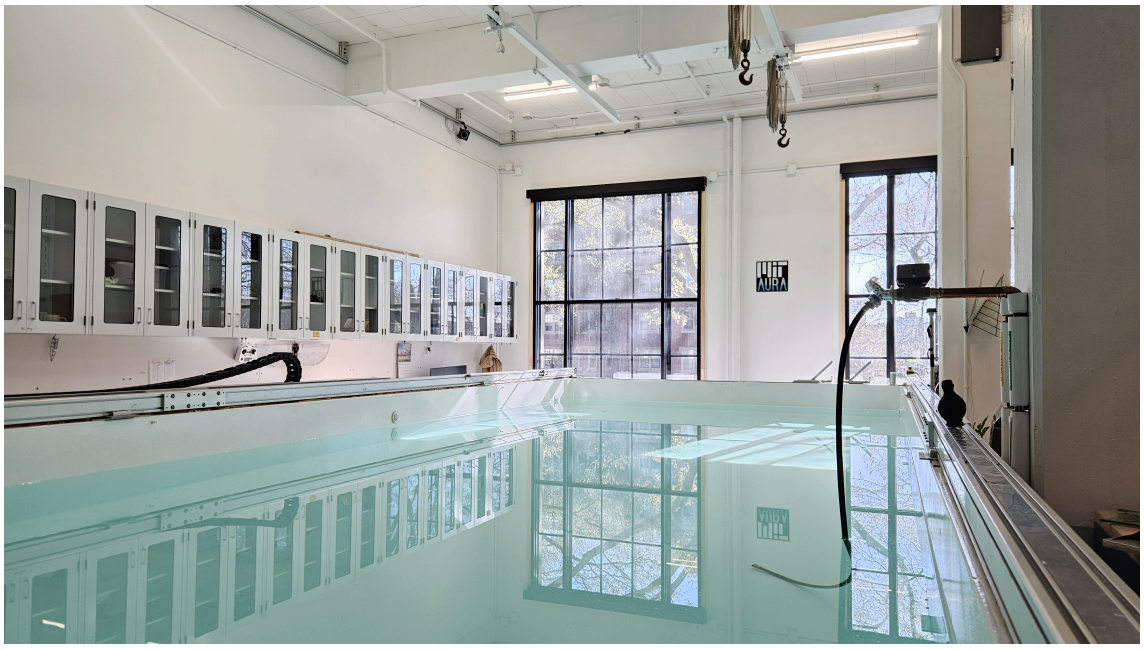


Figure 12.1: The AURA Lab tank. The rail and gantry run the length of the near side; the tracking cameras are mounted overhead.

⚠ Tank rules (repeat)

Nobody gets in the tank. Not to retrieve a robot, not to reposition it, not for a moment. Use the **poles** to fish the robot back to the edge.

Keep water off the floor. Water on the floor sets off an alarm, which ends the session for everyone. Lift the robot out over the tank and let it drain there rather than carrying it dripping, and keep **towels** to hand for what gets past. Dry the floor immediately if it happens.

12.5 What to log

Every run should record, at minimum:

- **Power consumption** — the numerator of cost of transport
- **IMU data** — attitude and acceleration through the run
- **Flapping frequency** — the input being varied

Since underwater we lose wireless communication, the data needs to be recorded and streamed back wirelessly after the run. There are two places to record data on the flight controller, and the choice matters more than it looks:

	RAM	Flash
Survives power loss	No	Yes
Speed	Fast, no wear	Slower, finite write cycles
Use it for	The normal case	Runs that cannot afford to be repeated

The normal workflow is **log to RAM during the run, then broadcast over BLE once the robot is out of the water**. Nothing has to be opened and nothing has to be plugged in, which is the whole point: the board is potted from Module 10 and there is no connector to reach.

⚠ RAM is lost the moment power goes

A brownout, a watchdog reset, or an accidental disconnect takes the whole run with it. Broadcast the data before changing the battery, not after - and if a run is expensive to repeat, write it to flash instead.

12.6 Housing

Design and 3D-print an exterior housing to ensure neutral buoyancy as needed. Foam can also be used.

12.7 Trim

Two independent adjustments, both mechanical. Change **one at a time** with a run between them, otherwise there is no telling which one did what.

12.7.1 Wing trim

Wing trim is adjusted at the **trailing edge**.

! To be completed by the instructor

The trailing-edge trim part itself is not designed yet — decide what it is, how it mounts, and what range of adjustment it gives. Until then this step cannot be written as a procedure, and students have nothing to adjust against.

12.7.2 Tail trim

Tail trim is manual: **bend and twist the tail by hand**. Bending sets pitch, twisting sets roll. Work in small increments and expect to go back and forth - there is no scale and no reference mark, so the logged runs are the only feedback available.

12.8 Cost of transport

Cost of transport is the score in Module 12, so measure it the same way every time:

$$\mathrm{COT} = \frac{P}{m\,g\,v} \tag{12.1}$$

with P the average electrical power over the run, m the robot's mass, and v its average forward speed. It is dimensionless, and lower is better.

The two halves come from two different places:

- **Power, onboard.** From the logged consumption above, averaged over the steady part of the run — not the acceleration at the start.
- **Speed, from the ceiling cameras.** The overhead tracking in Figure 12.1 gives position against time, which is the honest way to get v; timing it by eye across the tank is not good enough for a scored number.

Take both from the *same* window of the same run. Averaging power over one interval and speed over another is the most common way this number comes out wrong.

! TBC

Still missing from the protocol:

- **Run profile** — depth, duration, and the flap-frequency sweep to cover
- **Abort procedure** — what conditions end a run early, and what the student does at the tank edge when they do
- **Camera tracking, practically** — how a team starts a capture, and in what form the position data comes back

i Checkpoint

A completed underwater run with power, IMU and flapping frequency logged and successfully retrieved over BLE, no water ingress on inspection afterwards, and a cost-of-transport figure computed from Equation (12.1) with the camera-derived speed.

13. Module 12 — Competition

13.1 Summary

Robots are scored in two categories: **most efficient** and **fastest**.

13.2 The competition rules

13.3 Requirements

Bring	Robot, spare crank–linkage shaft, spare wing, charged packs, the logged cost-of-transport data

! TBC

Competition format is course material, not in the manufacturing guide. Needed:

- Course layout and run rules for each category
- How cost of transport is computed and normalised across teams with different masses
- Number of attempts, and repair rules between attempts
- How the competition contributes to the grade
- What happens to robots afterwards

13.4 Feedback

The final module is a recap and feedback round. This manual is rebuilt each year from what students report -if something in it cost time, say so. Corrections made during this semester are listed in the Changelog.

II Reference

14 Parts list 61
15 Brushless motors 63
16 The G431 ESC board 67
17 How SimpleFOC works 71
18 Software 75
19 Design notes 77
20 Instructor checklist 79
21 Changelog 83

14. Parts list

Category	Component	Material/Model	Per robot	Stock
Structure	Frame	Water-jet cut AL6061 1.5 mm	2	9 usable
	Vertical panels	Onyx with CF (Markforged)	2	
	Motor holder	Onyx with CF (Markforged)	1	12
	Battery holder	Onyx with CF (Markforged)	1	
	Tail holder	Onyx with CF (Markforged)	1	
	Screws	M2 Stainless	25	300
	Square nuts	M2 stainless	25	300
Propulsion	Crank	Al 7075	1	12
	Linkage	Al 7075	1	12
	Reduction	Series 20/1R 1:23 (Faulhaber)	1	PROD
	Right plate	Carbon 1.5mm plate	1	
	Left wing gear	Onyx with CF (Markforged)	1	
	Right wing gear	Onyx with CF (Markforged)	1	
	3x1 mm washer	AL	2	??
	Bearing (Link out)	3x8x4	1	14
	Bearing (Red out)	4x8x3 Flanged	1	14
	Bearing (Gears)	3x7x3 Flanged	4	52
	Bearing (Crank out)	4x10x4	1	14
Electronics	Flight controller	Seeeduino NRF52840	1	12
	Motor	AS2303 1500kV (T-Motors)	1	8
	Hall sensor	AS5047P	1	12

Category	Component	Material/Model	Per robot	Stock
	Hall sensor magnet	2.5mm diametric	1	40
	Cables signal	6x 30AWG, 1m	0.2	10
	Cable power	24 AWG, 1m	0.2	
Waterproof.	Seeeduino	Silicone + 3D print	1	Dow734
	ESC	Silicone + 3D print	1	Dow734
	Connector	SlimSeal TE	2	15
Battery	Battery	4S 380 mAh LiPo (Swaytronics)	1	??
Wings	Carbon rods		2	
	Membrane	Ripstop Nylon 40D, 48 g/m2, m2	1	15
	Carbon strips		2	
	Bonding	3M VHB tape + stitches at the ends	1	1

14.1

15. Brushless motors

Background for Module 2.

15.1 Start with the brushed DC motor

Every result you need for a brushless motor is easier to see in a brushed one first, because the two obey the same electrical physics. The difference is *how* the windings get switched, not what the machine does once they are.

A DC motor is described by two constants and one resistance. Spinning the rotor induces a back-EMF proportional to speed, and passing current through the windings produces torque proportional to current:

$$U_{\text{emf}} = \frac{\omega}{k_V} \qquad \tau = k_T\, I \tag{15.1}$$

These are the same k_V and k_T derived in Module 2 — inverses of each other, not independent choices. The supply voltage splits between the back-EMF and the ohmic drop across the winding resistance R:

$$U = \frac{\omega}{k_V} + R\, I \tag{15.2}$$

Substitute $I = \tau / k_T$ and rearrange for torque, and the whole behaviour of the motor falls out as a **straight line**:

$$\tau = k_T \frac{U}{R} - \frac{k_T}{k_V R}\,\omega \tag{15.3}$$

15.1.1 Reading the torque-speed line

That line is the motor's entire steady-state character, and it is fixed by the applied voltage. Two points define it:

- **Stall torque** $\tau_{\text{stall}} = k_T U / R$, at $\omega = 0$. The rotor is held still, there is no back-EMF, and the only thing limiting current is the winding resistance. This is the largest torque the motor can produce and also the fastest way to destroy it - see Thermal limits.
- **No-load speed** $\omega_0 = k_V U$, at $\tau = 0$. The rotor spins fast enough that the back-EMF nearly cancels the supply, so almost no current flows.

Everything the motor can do at that voltage lies on the line between them. Mechanical power is $P = \tau\omega$, the product of a falling line and a rising one, so it is a parabola peaking at **half the no-load speed** with half the stall torque. Efficiency peaks somewhere else entirely — much closer to ω_0, because the copper loss that dominates at low speed scales with I^2 while the useful output only scales with I.

(The web version of this page carries an interactive plot of these three curves. See the manual online.)

> **i** The useful consequence
>
> Raising the voltage shifts the line outwards without changing its slope: more speed *and* more torque, same trade-off between them. That is why speed control is voltage control, and why a "faster" motor at fixed voltage is a lower-torque one.

(Online, this section carries a voltage slider showing the line sweeping outwards.)

15.2 What changes with a brushless motor

In a brushed motor the commutator and brushes are a mechanical switch riding on the rotor: they physically reconnect the windings as it turns, so the current always flows in whichever coil is best placed to produce torque. It works without any electronics at all, and it wears out, arcs, and limits how fast the motor can safely spin.

A **BLDC** motor deletes that switch. The permanent magnets go on the rotor, the windings stay on the stator in three phases, and an external controller decides which phases to energise based on where the rotor currently is. The motor becomes inseparable from its driver — this is why Module 3 exists, and why the rotor position measurement matters so much.

What this does **not** change is Equation (15.3). The torque and speed constants have the same meaning, the line has the same shape, and you size the motor the same way. What changes is that the line is no longer the whole story:

	Brushed	Brushless
Commutation	Mechanical, self-acting	Electronic, needs rotor position
Torque–speed line	The real limit	The *ideal* limit, then clipped by the driver
Wear	Brushes are consumable	Bearings only
Speed ceiling	Brush arcing and bounce	Commutation rate and sensor lag
Torque ripple	Smoothed by many commutator segments	Depends on drive scheme (trapezoidal vs FOC)

15.2.1 The envelope, not the line

The straight line assumes the supply can deliver whatever current the equations ask for. It cannot. At stall, Equation (15.2) gives $I = U/R$, and with the sub-ohm winding resistance of a small outrunner that is a number in the hundreds of amps — far beyond both the ESC and the motor itself.

So the driver enforces a **current limit**, and because $\tau = k_T I$ that is directly a torque limit. The usable region becomes:

- a **constant-torque region** at low speed, flat at whatever current limit you set, because the ESC is holding current rather than letting the line dictate it;
- a **corner speed** where the back-EMF has risen enough that the supply can no longer push that current;
- above it, the original Equation (15.3) line, falling to ω_0.

The flat part is not physics, it is a decision. On our robot it is the decision that keeps the motor alive at the low speeds used for swimming, where the torque–speed line alone would happily draw enough current to cook the windings.

15.3 Thermal limits

Nothing in Equation (15.3) knows about heat, which is why the equations cheerfully predict operating points that will destroy the motor in seconds.

Almost all the loss you care about is **copper loss** in the windings:

$$P_{\text{copper}} = I^2 R \tag{15.4}$$

The square is what makes this the binding constraint. Doubling the torque doubles the current and **quadruples** the heating. A motor comfortable at its rated current will not survive twice that for long, even though the torque equation says it is only a factor of two.

Three practical consequences:

1. **Stall is the worst case, not the best.** At $\omega = 0$ there is maximum current and no rotation, so no airflow and no useful output — all the input power becomes heat in the windings. A motor held at stall can burn out without the shaft moving at all.

2. **Continuous and peak ratings are different numbers.** A peak rating assumes seconds; a continuous rating assumes thermal equilibrium. Flapping is a *reciprocating* load, so the current is far from constant — what matters for heating is the **RMS** current over the cycle, not the peak you see on the bench supply.
3. **Iron losses grow with speed.** Hysteresis and eddy-current losses in the stator scale with electrical frequency, so at the 1700 rad/s discussed in Module 4 the copper is no longer the only thing getting warm.

(Online, this section carries sliders for the ESC current limit and the motor's continuous rating, showing how the two clip the torque–speed line.)

> ⚠ Heat is cumulative and invisible
>
> Current is the only symptom you can see, and it appears instantly, while the temperature it causes lags behind by tens of seconds. A run that looks fine on the ammeter can still be depositing heat faster than the motor sheds it. When tuning in Module 4, stop and touch the motor.

15.4 Why an outrunner

Brushless motors come in two arrangements. An **inrunner** spins a magnet rotor inside a stationary stator, the familiar shape from a drill. An **outrunner** inverts it: the stator sits still in the middle and the magnets are carried on a bell that rotates around the outside. Ours is an outrunner — it is the bell you machine and press the shaft into in Module 2.

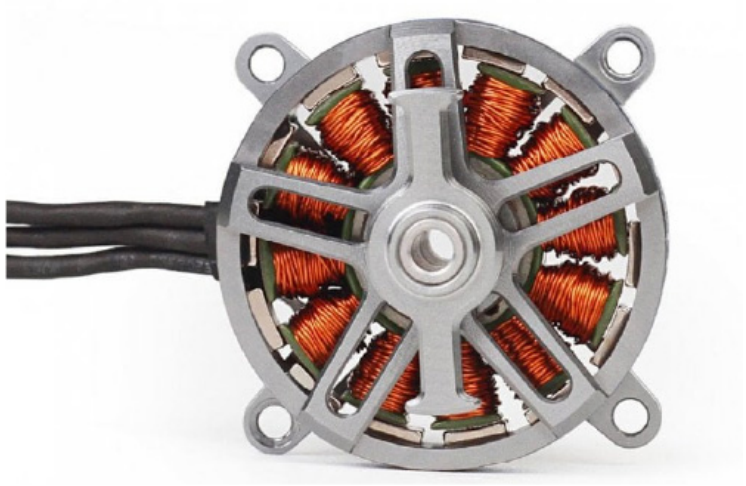


Figure 15.1: An outrunner seen from the back. The copper windings are the **stator** and do not move; the dark magnet segments around the rim are carried on the rotating bell. Note how far out the magnets sit — that radius is the lever arm in Equation (15.5), and it is the whole reason for this arrangement. Note also that the heat is generated in the middle, wrapped inside the part that spins.

The reason is torque. Torque is force times radius, and in an outrunner the magnets sit at the *largest* radius the motor's diameter allows, rather than the smallest:

$$\tau = F \times r \tag{15.5}$$

The same tangential force from the same windings acts on a longer lever arm. It also leaves room for more magnet poles around the circumference. Together these raise k_T for a motor of the same mass, which by Equation (15.1) means a **lower KV** — more torque per amp, less speed per volt.

That is the right trade for us. The wings need torque at low speed, through a gearbox that is already multiplying by 23.04:1, and the alternative is either a larger reduction or more current for the same result. Two secondary properties help as well:

- **Rotor inertia.** The rotating mass is at a large radius, so the bell has significant inertia and acts as a flywheel. Against a reciprocating flapping load that smooths the speed variation over a cycle. It also means the motor cannot change speed quickly, which is a real cost during transients.
- **Something to attach to.** The bell is a large-diameter rotating surface, which is what makes the shaft press-fit in Module 2 mechanically sensible in the first place.

The costs are real: outrunners spin more slowly for a given voltage, the exposed rotating bell is a hazard and an ingress path, and cooling is worse because the heat-producing stator is wrapped inside the rotor rather than mounted against the case. The last point ties directly back to Thermal limits — an outrunner's stator sheds heat mainly through its mount, which on our robot is a 3D-printed plate.

16. The G431 ESC board

Background on the **B-G431B-ESC1**, the board you rework and flash in Module 3. What is on it, how it turns a DC battery into three moving phases, what it can drive, and why it arrives in two pieces.

16.1 What it is

ST sells this as a *Discovery kit*: a reference design showing what their STM32G431 can do in a motor-control role, in the physical form of a drone ESC. That origin explains most of its oddities. A hobby ESC exposes a signal wire and two power leads because that is all a flight controller needs; this board brings out a CAN bus, a potentiometer input, a hall-sensor connector and a debug header, because its job is to demonstrate everything the chip supports.

For us the consequence is good and bad in equal measure. Good: the silicon is open, we can run our own firmware (SimpleFOC) and we get real current sensing. Bad: several connectors are wired for uses we do not want, which is why Module 3 spends its time repurposing them.

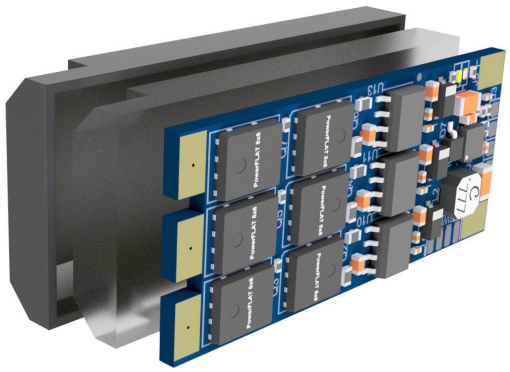


Figure 16.1: The power stage. The six large flat packages are the MOSFETs, in three pairs; the copper tabs on the left are the U, V and W phase outputs.

16.2 The parts that matter

Block	Part	Notes
Microcontroller	**STM32G431CB**	170 MHz Cortex-M4 with FPU, 128 kB flash, 32 kB RAM
Power switches	**6 × STL180N6F7**	60 V N-channel MOSFETs, very low on-resistance
Gate drivers	**L6387**	Three half-bridge drivers, one per phase
Current sensing	**Three shunts**	Low-side, amplified into the MCU's ADCs
Programmer	**ST-LINK/V2-1**	On the detachable daughter-board

Sources: ST's product page and data brief.

The MCU is the interesting choice. FOC recomputes the field orientation every control loop, and each iteration costs a handful of trigonometric evaluations (see How SimpleFOC works). At 170 MHz with hardware floating point that fits comfortably inside a 10 kHz loop. A hobby ESC's 8-bit microcontroller could not do this arithmetic in the time available, which is precisely why hobby ESCs use the cruder back-EMF scheme instead.

16.3 How the three phases are generated

The battery supplies one DC voltage. The motor needs three voltages that vary smoothly and stay 120° apart. The bridge between the two is built from three identical circuits.

One half-bridge per phase. Each phase output connects to two MOSFETs in series across the battery: a **high-side** switch to the positive rail and a **low-side** switch to ground. Closing the high side pulls that phase to battery voltage; closing the low side pulls it to zero. Three phases, three pairs, six transistors — the six packages in Figure 16.1.

PWM makes the intermediate values. A switch is either on or off, so a half-bridge can only output the full rail or nothing. Switching between them tens of thousands of times a second and varying the *fraction* of time spent high produces any average voltage in between. 50% duty gives half the rail. The motor's own winding inductance does the smoothing: it resists rapid current changes, so it responds to the average and largely ignores the switching.

Three of them, offset, make a rotating field. Drive the three duty cycles as sinusoids 120° apart and the three phase voltages produce a magnetic field that sweeps steadily around the stator. Advance the sinusoids faster and the field rotates faster. This is what `driver.setPwm(Ua, Ub, Uc)` at the bottom of SimpleFOC's inner loop is ultimately setting.

> ⚠ Both switches in a pair must never be on together
>
> High and low side closed at once is a direct short across the battery through two transistors — called **shoot-through**, and it destroys the pair in microseconds. The hardware inserts a **dead time**: a brief interval where both are off during every transition. It is handled by the gate drivers and the MCU's timer, so you will not code it, but it is why the two switches of a pair are never driven by a single signal.

Current sensing closes the loop. A small-value **shunt resistor** sits in each phase's low-side return path. The voltage across it is proportional to the current through that phase; the MCU amplifies and samples it. Three shunts mean all three phase currents are visible, which is what makes true FOC current control possible rather than voltage-mode approximation. Without this measurement you can ask for a torque; with it you can *regulate* one.

16.4 What it can drive

Input voltage	**3S to 6S** LiPo (roughly 11–25 V)
Peak current	**40 A**, with forced-air cooling
Protection	Overcurrent, overvoltage and thermal, on-board

Two cautions about that 40 A. It is a **peak** figure, and it assumes **forced air** — a fan, or a drone's propwash. Our robot has neither, and it spends much of its life underwater with the board sealed inside potting compound (Module 10), so heat leaves slowly and only by conduction. Treat 40 A as a number describing the silicon, not a budget you may spend.

In practice this is not the binding constraint. The wing motor draws far less than the board can supply, so the limits you will actually meet are the motor's own thermal limit and sensor lag, not the ESC's. The relevant discussion is in Brushless motors and Module 4.

16.5 Why there is a daughterboard

The small board at the end is a complete **ST-LINK/V2-1** programmer and debug probe. It is separated from the power board by a line of perforations and is meant to be snapped off.

That design makes sense for ST's intended user: develop with the programmer attached, then break it away so the flight-ready ESC is smaller and lighter, and reuse the same programmer on the next board. It carries two things:

- **SWD** — the debug and programming interface, used to flash the MCU
- **A virtual COM port** — a USB serial bridge onto one of the MCU's UARTs

> ! Do not snap it off
>
> Once separated, reprogramming needs an external programmer wired to the SWD pads by hand. We keep it attached for the whole course. This is also why the board is larger than a hobby ESC of equivalent power — roughly half of what you are holding is the programmer.

The virtual COM port is where Module 3 runs into trouble. Out of the box it is wired to the MCU's **USART2**, on pins that the board also uses for the hall-sensor connector. Those same pins are the SPI3 bus we need for the AS5047P encoder, and an encoder is not optional for FOC. So the pins go to the encoder, USART2 stops being reachable over USB, and serial has to be re-routed — either onto USART1 by rework, or off-board entirely to the Seeeduino. Flashing is unaffected throughout, because SWD runs on different pins.

16.6 What we do not use

- **CAN transceiver.** Present and functional; our robot uses BLE from the flight controller instead.
- **Potentiometer input.** An analog throttle knob for bench demonstrations.
- **Hall-sensor connector (J8).** Intended for the three digital hall sensors built into many BLDC motors. Ours has none, and hall sensors would be too coarse anyway — six positions per electrical revolution, against the AS5047P's continuous absolute angle. The connector gets repurposed for serial.

Sources: ST B-G431B-ESC1 product page, data brief.

17. How SimpleFOC works

The firmware in FAAV-G431-ESC is a SimpleFOC application. This page is what you need to read that firmware and change it with confidence: what the library computes, in what order, and where your settings enter. It is background for Module 3 and Module 4, not a procedure.

17.1 The problem it solves

A brushless motor has three windings and no brushes, so nothing decides on its own which winding should be energised. Something has to look at where the rotor is and energise the windings to put the stator's magnetic field about **90° ahead** of the rotor's magnets, the orientation that produces the most torque per amp. Do that continuously and the motor behaves like a brushed DC motor: torque proportional to current, no commutator to wear out. That is **Field-Oriented Control**, and it is why Module 2's $\tau = k_T I$ is usable at all.

The awkward part is that "90° ahead" is a statement about a *rotating* frame, while the three phase currents live in a *stationary* one. FOC is mostly the bookkeeping that moves between the two.

17.2 Three objects, and what links them

The SimpleFOC architecture is built around three objects, each wrapping one piece of hardware:

Object	Wraps	Our hardware
`MagneticSensorSPI`	The angle sensor	AS5047P on SPI3
`BLDCDriver6PWM`	The inverter	Six MOSFETs on the G431
`BLDCMotor`	The control maths	—

`BLDCMotor` owns the algorithm and is told about the other two with `linkSensor()` and `linkDriver()`. Nothing else connects them, which is why a missing `link` call produces a motor that initialises cleanly and then does nothing sensible.

17.3 The two loops

Every iteration runs **two nested controllers**, and keeping them straight is most of understanding the library:

- `motor.loopFOC()` — the **inner** loop. Reads the angle, works out where the field should point, and sets the PWM duty cycles. It runs as fast as you can call it (above 10 kHz), and its rate is the loop frequency you read back with the commander's `F` command.
- `motor.move(target)` — the **outer** loop. Turns your setpoint (a position, a speed, or a torque) into the q-axis demand the inner loop chases.

The inner loop is where speed matters; the outer loop is where tuning matters.

> ⚠ Both must be called every iteration
>
> `loopFOC()` without `move()` holds whatever demand it last had. `move()` without `loopFOC()` updates a number nothing acts on. Anything blocking in `loop()` - a `delay()`, a slow `print` - drops the inner-loop rate and shows up as a motor that runs rough at speed.

17.4 What the inner loop actually computes

Distilled from the library, with the bookkeeping removed:

```
void loopFOC() {
  sensor.update();

  // 1. Mechanical angle -> electrical angle.
  //    A motor with p pole pairs turns through p electrical revolutions
  //    for every mechanical one. zero_electric_angle comes from initFOC().
  float angle_el = normalizeAngle(sensor.getAngle() * pole_pairs
                                  - zero_electric_angle);

  switch (torque_controller) {

    case TorqueControlType::voltage:
      // No current measurement. Ask for a voltage and hope; cheap and
      // adequate at low speed. This is where you start.
      Uq = target_voltage_q;
      Ud = 0;
      break;

    case TorqueControlType::foc_current:
      // 2. Measure two phase currents; the third follows from ia+ib+ic = 0.
      PhaseCurrent_s c = current_sense.getPhaseCurrents();

      // 3. Clarke (abc -> alpha/beta) then Park (alpha/beta -> d/q),
      //    using angle_el. The output is two DC values, not sinusoids.
      DQCurrent_s i = current_sense.getFOCCurrents(angle_el);

      // 4. Two independent PI controllers on those DC values.
      //    q is torque-producing; d is pure flux and is driven to zero
      //    because it costs current and produces nothing.
      Uq = PID_current_q(target_current_q - i.q);
      Ud = PID_current_d(0.0f - i.d);
      break;
  }

  setPhaseVoltage(Uq, Ud, angle_el);
}
```

The Park transform is the trick worth internalising. In the stationary frame the phase currents are three sinusoids and there is nothing steady to control. Rotating the frame with the rotor turns them into **two constants**, i_d and i_q, and a constant is something an ordinary PI controller can hold. Then you rotate back:

```
void setPhaseVoltage(float Uq, float Ud, float angle_el) {
  // Inverse Park: back to the stationary frame
  float Ualpha = Ud * cos(angle_el) - Uq * sin(angle_el);
  float Ubeta  = Ud * sin(angle_el) + Uq * cos(angle_el);

  // Inverse Clarke: two axes back to three phases
  float Ua = Ualpha;
  float Ub = -0.5f * Ualpha + _SQRT3_2 * Ubeta;
  float Uc = -0.5f * Ualpha - _SQRT3_2 * Ubeta;

  // Shift to sit around mid-rail, since the inverter cannot output
  // a negative voltage, then hand duty cycles to the hardware.
  driver.setPwm(Ua + Udc/2, Ub + Udc/2, Uc + Udc/2);
}
```

Setting $U_d = 0$ and driving only U_q is exactly "put the field 90° ahead", expressed in coordinates. Everything above is the machinery for saying it in terms the three phases understand.

17.5 What the outer loop computes

`move()` is a cascade, and which stages run depends on `motor.controller`:

```
void move(float target) {
  switch (controller) {
    case MotionControlType::angle:
      // position -> velocity -> current, all three stages
      shaft_velocity_sp = P_angle(target - shaft_angle);
      target_current_q  = PID_velocity(shaft_velocity_sp - shaft_velocity);
      break;

    case MotionControlType::velocity:
      target_current_q  = PID_velocity(target - shaft_velocity);
      break;

    case MotionControlType::torque:
      target_current_q  = target;   // straight through
      break;
  }
}
```

This is why the commander's `T` means something different in each mode — it is the input to whichever stage is outermost. It is also why tuning proceeds inside-out in Module 4: the velocity PI has to be right before the position P above it can be.

17.6 Alignment, and why the motor twitches at power-on

`motor.initFOC()` performs the one calibration FOC cannot do without. The sensor reports *some* absolute angle, but nothing tells it how that number relates to the motor's magnetic poles - that depends on how the magnet was glued on and how the phase wires happened to be soldered. So `initFOC()` drives a known field, watches which way the rotor moves and where it settles, and stores two things:

- **`zero_electric_angle`** - the offset between sensor zero and electrical zero
- **sensor direction** - whether the sensor counts up or down relative to the motor's positive rotation

That is the twitch you see at every power-on. If it is skipped or wrong, the field is placed at the wrong angle and the motor draws current while producing little torque, or runs away.

> **i** Why the UVW order does not matter
>
> Module 3 says you can solder the phase wires in any order. Swapping two phases reverses the motor's direction, and `initFOC()` measures direction rather than assuming it - so the calibration simply comes out with the other sign. The cost of getting it "wrong" is a stored constant, not a rework.

17.7 The main ESC code

Setup is configuration; the loop is two calls. Trimmed to the parts that matter:

```
#include <SimpleFOC.h>

BLDCMotor motor = BLDCMotor(7);                 // pole pairs
BLDCDriver6PWM driver = BLDCDriver6PWM(A_PHASE_UH, A_PHASE_UL,
                                       A_PHASE_VH, A_PHASE_VL,
                                       A_PHASE_WH, A_PHASE_WL);
MagneticSensorSPI sensor = MagneticSensorSPI(AS5047_SPI, PA15);

float target = 0;

void setup() {
```

```
  sensor.init();
  motor.linkSensor(&sensor);

  driver.voltage_power_supply = 14.8;
  driver.init();
  motor.linkDriver(&driver);

  motor.foc_modulation = FOCModulationType::SpaceVectorPWM;
  motor.controller     = MotionControlType::velocity;

  motor.PID_velocity.P = 0.2;
  motor.PID_velocity.I = 20;
  motor.LPF_velocity.Tf = 0.01;   // velocity is differentiated, so filtered
  motor.voltage_limit   = 6;      // the safety net -- set it low first

  motor.init();
  motor.initFOC();                // alignment happens here
}

void loop() {
  motor.loopFOC();
  motor.move(target);
}
```

The pole-pair count must match the motor or every angle in the chain is wrong by a fixed ratio, and the symptom is a motor that vibrates instead of turning. `voltage_limit` is the one setting to get right before the first spin-up: it caps what the inner loop can ask for regardless of what the outer loop demands.

17.8 Where to look next

- SimpleFOC communication docs — the commander interface used all through Module 3
- Commander motor reference — every letter the commander accepts
- The G431 ESC board — what `driver.setPwm()` is actually driving

18. Software

18.1 ESC

Code for the B-G431B-ESC1 motor controller using the SimpleFOC library: FAAV-G431-ESC. You need access to the github folder.

Outdated versions, kept for reference:

- Old EPFL code for NRF52: FAAV-Seeeduino
- Old MIT code for NRF52/ESP32: FAAV-AUK

18.2 External documentation

- SimpleFOC communication
- SimpleFOC commander
- XIAO ESP32S3 getting started

19. Design notes

Why the robot is built the way it is and latest changes

19.1 Log of 2026 changes

- Front of gears: exactly 1 mm spacing for washer. Back of gears (towards tail) has builtin 3D-printed spacers that press against inner bearing. Printing them will not use support
- All 3D printed cross fixing is now done with M2 square half nuts (stainless). Takes less space and less work than heat inserts. 3D printed plates are all 4 mm thick.
- Tail tube is now 5/3 round instead of square. The bracket (tail rigidifier) adapted in consequence.
- Left gear extended and wraps around the linkage for better support.
- Crank-link shaft is now 10.5 mm to have enough space to fit a circlip.
- Linkage bearings are now 4x10x4 on input and 3x8x4 on output. MAX static loading of 130 N.
- ESC box is now screwed against right bearing holder
- Front dome mounts to front plate
- Micro-controller box: to be printed transparent, no more window. Square nut assembly. USB-C port sits closer to edge of part, should give a better connection to program.
- Gear shafts bearings changed to 3x7x3 flanged, with the flange now inserting into part.
- All parts relabeled

19.2 Gearbox corrosion

By default the gearbox on the FAAV4 is not waterproof at all, and it is normal for it to fill completely with water once submerged. This is considered acceptable: over more than a year of repeated use in water, including in lakes, a gearbox was opened and showed no sign of corrosion.

Proper lubrication: a mix of **CorrosionX and white lithium grease** - help with corrosion problems. The practical procedure is in Module 10.

20. Instructor checklist

Stock levels and order status are in the Parts list.

20.1 Module 1 — Orientation

Module page

- ☐ Class cabinet labelled
- ☐ Safety goggles in the “safety” drawer
- ☐ Prepare safety briefing with the safety officer (Ethan Fulcher)

20.2 Module 2 — Motor and transmission shaft

Module page

- ☐ 3D prints
- ☐ Clean CAD
- ☐ Prepare solder kits

20.3 Module 3 — Motor driver, sensor and serial connection

Module page

Stage (per team)

- ☐ AS5047P encoder and its 3D-printed mount
- ☐ 6×30AWG wire ribbon
- ☐ Seeeduino XIAO nRF52840 Sense and its 3D-printed bracket
- ☐ 3D-printed ESC bracket
- ☐ Soldering iron, torx drivers, tweezers, micro-USB cable
- ☐ **Power source, 15 V 5 A, over USB-C**

Prepare in advance

- ☐ Print the encoder, Seeeduino and ESC brackets
- ☐ Calibration stand and calibration code
- ☐ Barebone Seeeduino sketch ready to clone
- ☐ Students told to install VS Code + PlatformIO before the session

20.4 Module 4 — Closed-loop control and tuning

Module page

Stage (per team)

- ☐ Faulhaber 23.04:1 gearbox
- ☐ 3D-printed plates and the second vertical bracket
- ☐ M2.5×10 screws, 3 × M2.5 and 3 × M2 screws
- ☐ 4×8 mm flanged bearing
- ☐ 3 wires for the Seeeduino link
- ☐ Bench supply with current limit
- ☐ Tachometer or camera
- ☐ Soldering iron
- ☐ Material for a small flag on the gearbox output

Prepare in advance

- ☐ **Pre-flash every ESC** with FAAV-G431-ESC, including the SPI3 pin remap

Open items

- ☐ Say which repository and branch hold the barebone Seeeduino sketch. The nRF52 repositories in Software are marked outdated
- ☐ Write the Calibration section, which is currently empty
- ☐ Write the PID tuning exercise, the target performance and the sign-off criteria

20.5 Module 5 — Frame, crank, linkage and gears

Module page

Stage (per team)

- ☐ Carbon frame plates (1.5 mm, top and bottom), M2×4 screws and brass inserts
- ☐ AL7075 crank and linkage
- ☐ 4 mm crank–linkage shaft cut to **10.5 mm**, and a 4 mm circlip
- ☐ 3 mm gear shafts
- ☐ Left and right wing gears
- ☐ Bearings: 10×4×4 (crank side), 8×3×4 (wing side), 8×4×4 flanged (middle plate), 4 × 7×3×3 flanged (wing pivots)
- ☐ M2×5 screws
- ☐ Drill press, metal glue, caliper

Prepare in advance

- ☐ CNC-cut the carbon plates
- ☐ CNC-cut the crank and linkage in AL7075
- ☐ Cut the 4 mm shafts to 10.5 mm
- ☐ Print the gears
- ☐ **Buy 4 mm circlips** (still a TODO on the module page)

20.6 Module 6 — Flapping simulation (Ptera)

Module page

Stage

- ☐ Students bring laptops with Ptera installed

Open items

This module has no counterpart in the manufacturing guide, so most of it still needs writing:

- ☐ Ptera installation instructions and a worked example
- ☐ The motor power curve students are given, and its file format
- ☐ How “most efficient thrust profile” is defined and scored
- ☐ Buildability constraints: span limits, root fitting, mass budget
- ☐ How far teams may deviate from the reference wing

20.7 Module 7 — Wing fabrication

Module page

Stage (per team)

- ☐ Carbon rods and strips
- ☐ Carbon wing roots, drilled to mate with the gears
- ☐ 2 × nylon adapters per wing (4 per robot)
- ☐ Ripstop nylon 40D membrane
- ☐ 3M VHB tape
- ☐ Superglue
- ☐ Thread and needles
- ☐ Cutting mats, steel rules, scalpels
- ☐ Scale, for matching the mass of the two wings

- ☐ A printer that can print the team's Module 6 geometry at 1:1

Prepare in advance

- ☐ Drill the wing roots
- ☐ Print the nylon adapters

Open items

- ☐ 3M tape: which VHB product, what width, and where it goes on the leading edge
- ☐ Spar layup: rod and strip diameters, and where the diagonals attach
- ☐ Thread type and stitch spacing
- ☐ The allowed mass and geometry difference between left and right wings

20.8 Module 8 — Tail

Module page

Stage (per team)

- ☐ Carbon sheet and tail rod
- ☐ 5 mm tail tube
- ☐ Foam stock (XPS or closed-cell PE)
- ☐ Epoxy
- ☐ M2 screws and square half nuts, for the trailing-edge holder collar
- ☐ Foam cutter, scalpel
- ☐ 3D printer access for the students' trailing-edge holder designs

Open items

- ☐ Tail sheet material and thickness, rod diameter, boom mount, and incidence setting
- ☐ Decide whether tail actuation is in scope (if time permits)
- ☐ Say which print material the trailing-edge holder should use

20.9 Module 9 — Flight controller, PCB and dry testing

Module page

Stage (per team)

- ☐ Seeeduino XIAO, connectors, battery
- ☐ Hotplate, tweezers, multimeter
- ☐ Bench supply with current limit

Prepare in advance

- ☐ Custom PCBs from FAAV-PCB-v3 and their BOM components
- ☐ FAAV-Flight-Controller firmware checked against the hardware in use. Software lists it as ESP32, while Modules 3 and 9 use the XIAO nRF52840
- ☐ Charged batteries

Open items

- ☐ Write the "Connect battery" instructions
- ☐ Dry-test protocol: the mount used while flapping in air, the frequency sweep, what gets measured, the wireless control setup, and abort criteria

20.10 Module 10 — Waterproofing

Module page

> ⚠ Schedule for the cure
>
> Potting takes 12–24 h to cure. Plan the session so potting happens first, or split it across two sittings.

Stage (per team)

- ☐ 3D-printed flight controller waterproofing box (printed transparent)
- ☐ 3D-printed ESC potting frame
- ☐ Silicone sealant (Dow 734) for the flight controller
- ☐ Silcoset for the ESC
- ☐ Kapton tape
- ☐ USB-C cable, to plug the port during potting
- ☐ CorrosionX and white lithium grease
- ☐ SlimSeal TE connectors (2 per robot)
- ☐ IPA for cleaning after immersion
- ☐ Needle, soldering iron
- ☐ Setup for the static immersion test

Prepare in advance

- ☐ Print the waterproofing boxes and ESC frames
- ☐ Somewhere the potted parts can cure undisturbed overnight

Open items

- ☐ Battery waterproofing procedure
- ☐ Battery connector waterproofing procedure
- ☐ The discussion of cost of transport and how it is measured

20.11 Module 11 — Wet testing

Module page

Stage

- ☐ Tank ready, with retrieval poles and towels
- ☐ Overhead tracking cameras working
- ☐ Charged 4S packs
- ☐ Spare crank–linkage shafts
- ☐ Floats or foam for trimming buoyancy
- ☐ Scale, for the robot mass used in cost of transport

Prepare in advance

- ☐ A way for teams to print housings for neutral buoyancy
- ☐ BLE retrieval of logged runs tested end to end

Open items

- ☐ Design the trailing-edge trim part: what it is, how it mounts, and its range of adjustment
- ☐ Run profile: depth, duration, and the flap-frequency sweep
- ☐ Abort procedure: what ends a run, and what students do at the tank edge
- ☐ Camera tracking: how a team starts a capture, and the format the position data comes back in

20.12 Module 12 — Competition

Module page

Stage

- ☐ Tank and tracking cameras, set up as in Module 11
- ☐ Spare crank–linkage shafts, spare wing material, charged packs

Open items

- ☐ Write the competition rules section, which is currently empty
- ☐ Course layout and run rules for each category (most efficient, fastest)
- ☐ How cost of transport is computed and normalised across robot masses
- ☐ Number of attempts, and repair rules between attempts
- ☐ How the competition counts towards the grade
- ☐ What happens to the robots afterwards

21. Changelog

Corrections and additions published during the semester, newest first.

If you are working from a printed copy, check this page before each module. Changes that affect a step you may already have completed are marked in **bold**.

> For maintainers
>
> One row per change: date, page, what changed, and whether teams need to redo anything. Do not edit a step silently — a dated entry is what lets a student at the bench trust the rest of the page.

21.1 fall-2026

Date	Page	Change	Action needed
—	—	Manual published for the Fall 2026 cohort, derived from *FAAV Manufacturing Guide* v4.0	—